```
!wget https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/002/492/original/ola_driver_scaler.csv
```

```
--2024-10-17 06:12:43--  https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/002/492/original/ola_driver_scal
Resolving d2beiqkhq929f0.cloudfront.net (d2beiqkhq929f0.cloudfront.net)... 13.224.9.129, 13.224.9.181, 13.224.9.24, ...
Connecting to d2beiqkhq929f0.cloudfront.net (d2beiqkhq929f0.cloudfront.net)|13.224.9.129|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1127673 (1.1M) [text/plain]
Saving to: 'ola_driver_scaler.csv'

ola_driver_scaler.c 100%[===================>]   1.08M  1014KB/s    in 1.1s

2024-10-17 06:12:45 (1014 KB/s) - 'ola_driver_scaler.csv' saved [1127673/1127673]
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
df= pd.read_csv('ola_driver_scaler.csv')
pd.set_option('display.max_columns', None)
df
```

	Unnamed: 0	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designatio
0	0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	
1	1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	
2	2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	
3	3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	
4	4	12/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	
...	...	...	...	...	...	...	...	...	...	...	
19099	19099	08/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	
19100	19100	09/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	
19101	19101	10/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	
19102	19102	11/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	
19103	19103	12/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19104 entries, 0 to 19103
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Unnamed: 0            19104 non-null  int64
 1   MMM-YY                19104 non-null  object
 2   Driver_ID             19104 non-null  int64
 3   Age                   19043 non-null  float64
 4   Gender                19052 non-null  float64
 5   City                  19104 non-null  object
 6   Education_Level       19104 non-null  int64
 7   Income                19104 non-null  int64
 8   Dateofjoining         19104 non-null  object
 9   LastWorkingDate       1616 non-null   object
 10  Joining Designation   19104 non-null  int64
 11  Grade                 19104 non-null  int64
 12  Total Business Value  19104 non-null  int64
 13  Quarterly Rating      19104 non-null  int64
dtypes: float64(2), int64(8), object(4)
memory usage: 2.0+ MB
```

```
df[df['Joining Designation']!=df['Grade']]
```

	Unnamed: 0	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designatio
28	28	01/01/19	13	29.0	0.0	C19	2	119227	28/05/15	NaN	1
29	29	02/01/19	13	29.0	0.0	C19	2	119227	28/05/15	NaN	1
30	30	03/01/19	13	29.0	0.0	C19	2	119227	28/05/15	NaN	1
31	31	04/01/19	13	29.0	0.0	C19	2	119227	28/05/15	NaN	1
32	32	05/01/19	13	29.0	0.0	C19	2	119227	28/05/15	NaN	1
...	...	...	...	...	...	...	...	...	...	...	..
19074	19074	08/01/20	2784	34.0	0.0	C24	0	82815	15/10/15	NaN	2
19075	19075	09/01/20	2784	34.0	0.0	C24	0	82815	15/10/15	NaN	2
19076	19076	10/01/20	2784	34.0	0.0	C24	0	82815	15/10/15	NaN	2
19077	19077	11/01/20	2784	34.0	0.0	C24	0	82815	15/10/15	NaN	2
19078	19078	12/01/20	2784	34.0	0.0	C24	0	82815	15/10/15	NaN	2

```
df.drop(['Unnamed: 0'],axis=1,inplace=True)
df
```

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	B
0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	
1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	
2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	1	1	
3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	2	
4	12/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	2	
...	...	...	...	...	...	...	...	...	...	...	...	...
19099	08/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19100	09/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19101	10/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19102	11/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19103	12/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df.nunique()
```

	0
MMM-YY	24
Driver_ID	2381
Age	36
Gender	2
City	29
Education_Level	3
Income	2383
Dateofjoining	869
LastWorkingDate	493
Joining Designation	5
Grade	5
Total Business Value	10181
Quarterly Rating	4

dtype: int64

```
df[df['Driver_ID']==2788]
```

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	B
19097	06/01/20	2788	29.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19098	07/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19099	08/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19100	09/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19101	10/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	
19102	11/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	

```
df.isna().sum()
```

	0
MMM-YY	0
Driver_ID	0
Age	61
Gender	52
City	0
Education_Level	0
Income	0
Dateofjoining	0
LastWorkingDate	17488
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0

dtype: int64

```
df.rename(columns={'MMM-YY':'Reportingdate'},inplace=True)


# Converting date time related features to datetime datatypes

#df['Reportingdate']=pd.to_datetime(df['Reportingdate'])
#df['Dateofjoining']=pd.to_datetime(df['Dateofjoining'])
#df['LastWorkingDate']=pd.to_datetime(df['LastWorkingDate'])


df_original=df.copy(deep=True)


# Creating a new column 'Target' based on LastWorkingDate to check if the driver
# is still with OLA or not.

df['Target']=np.where(df['LastWorkingDate'].isna(),0,1)
df
```

	Reportingdate	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	G
0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	
1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	
2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	1	
3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	
4	12/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	
...	...	...	...	...	...	...	...	...	...	...	...
19099	08/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	
19100	09/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	
19101	10/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	
19102	11/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	
19103	12/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
# Converting date time related features to datetime datatypes

df['Reportingdate']=pd.to_datetime(df['Reportingdate'])
df['Dateofjoining']=pd.to_datetime(df['Dateofjoining'])
df['LastWorkingDate']=pd.to_datetime(df['LastWorkingDate'])


# df.drop('LastWorkingDate',axis=1,inplace=True)
# df
```

```
df.isna().sum()
```

	0
Reportingdate	0
Driver_ID	0
Age	61
Gender	52
City	0
Education_Level	0
Income	0
Dateofjoining	0
LastWorkingDate	17488
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0
Target	0

dtype: int64

```
df_nums=df.select_dtypes(np.number)
df_nums
```

	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Target
0	1	28.0	0.0	2	57387	1	1	2381060	2	0
1	1	28.0	0.0	2	57387	1	1	-665480	2	0
2	1	28.0	0.0	2	57387	1	1	0	2	1
3	2	31.0	0.0	2	67016	2	2	0	1	0
4	2	31.0	0.0	2	67016	2	2	0	1	0
...	...	...	...	...	...	...	...	...	...	...
19099	2788	30.0	0.0	2	70254	2	2	740280	3	0
19100	2788	30.0	0.0	2	70254	2	2	448370	3	0
19101	2788	30.0	0.0	2	70254	2	2	0	2	0
19102	2788	30.0	0.0	2	70254	2	2	200420	2	0
19103	2788	30.0	0.0	2	70254	2	2	411480	2	0

Next steps: Generate code with df_nums View recommended plots New interactive sheet

```
df_nums.drop(['Driver_ID'],axis=1,inplace=True)
df_nums
```

	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Target
0	28.0	0.0	2	57387	1	1	2381060	2	0
1	28.0	0.0	2	57387	1	1	-665480	2	0
2	28.0	0.0	2	57387	1	1	0	2	1
3	31.0	0.0	2	67016	2	2	0	1	0
4	31.0	0.0	2	67016	2	2	0	1	0
...	...	...	...	...	...	...	...	...	...
19099	30.0	0.0	2	70254	2	2	740280	3	0
19100	30.0	0.0	2	70254	2	2	448370	3	0
19101	30.0	0.0	2	70254	2	2	0	2	0
19102	30.0	0.0	2	70254	2	2	200420	2	0
19103	30.0	0.0	2	70254	2	2	411480	2	0

19104 rows × 9 columns

Next steps: Generate code with df_nums View recommended plots New interactive sheet

```
# Using KNN Imputer to fill missing values

from sklearn.impute import KNNImputer
imputer=KNNImputer(n_neighbors=5,weights='distance')
df_new=pd.DataFrame(imputer.fit_transform(df_nums),columns=df_nums.columns)
df_new
```

	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Target
0	28.0	0.0	2.0	57387.0	1.0	1.0	2381060.0	2.0	0.0
1	28.0	0.0	2.0	57387.0	1.0	1.0	-665480.0	2.0	0.0
2	28.0	0.0	2.0	57387.0	1.0	1.0	0.0	2.0	1.0
3	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	0.0
4	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	0.0
...	...	...	...	...	...	...	...	...	...
19099	30.0	0.0	2.0	70254.0	2.0	2.0	740280.0	3.0	0.0
19100	30.0	0.0	2.0	70254.0	2.0	2.0	448370.0	3.0	0.0
19101	30.0	0.0	2.0	70254.0	2.0	2.0	0.0	2.0	0.0
19102	30.0	0.0	2.0	70254.0	2.0	2.0	200420.0	2.0	0.0
19103	30.0	0.0	2.0	70254.0	2.0	2.0	411480.0	2.0	0.0

19104 rows × 9 columns

Next steps: Generate code with df_new View recommended plots New interactive sheet

```
df_new.isna().sum()
```

	0
Age	0
Gender	0
Education_Level	0
Income	0
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0
Target	0

dtype: int64

```
# Getting remaining columns other than numerical columns and concatinating back to the dataframe

df_other= df.select_dtypes(exclude=np.number)
df_other
```

	Reportingdate	City	Dateofjoining	LastWorkingDate
0	2019-01-01	C23	2018-12-24	NaT
1	2019-02-01	C23	2018-12-24	NaT
2	2019-03-01	C23	2018-12-24	2019-03-11
3	2020-11-01	C7	2020-11-06	NaT
4	2020-12-01	C7	2020-11-06	NaT
...	...	...	...	...
19099	2020-08-01	C27	2020-06-08	NaT
19100	2020-09-01	C27	2020-06-08	NaT
19101	2020-10-01	C27	2020-06-08	NaT
19102	2020-11-01	C27	2020-06-08	NaT
19103	2020-12-01	C27	2020-06-08	NaT

19104 rows × 4 columns

Next steps: Generate code with df_other View recommended plots New interactive sheet

```
df=pd.concat([df_original['Driver_ID'],df_new,df_other],axis=1)
df
```

	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Target	Reportingdate	City
0	1	28.0	0.0	2.0	57387.0	1.0	1.0	2381060.0	2.0	0.0	2019-01-01	C23
1	1	28.0	0.0	2.0	57387.0	1.0	1.0	-665480.0	2.0	0.0	2019-02-01	C23
2	1	28.0	0.0	2.0	57387.0	1.0	1.0	0.0	2.0	1.0	2019-03-01	C23
3	2	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	0.0	2020-11-01	C7
4	2	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	0.0	2020-12-01	C7
...	...	...	...	...	...	...	...	...	...	...	...	...
19099	2788	30.0	0.0	2.0	70254.0	2.0	2.0	740280.0	3.0	0.0	2020-08-01	C27
19100	2788	30.0	0.0	2.0	70254.0	2.0	2.0	448370.0	3.0	0.0	2020-09-01	C27
19101	2788	30.0	0.0	2.0	70254.0	2.0	2.0	0.0	2.0	0.0	2020-10-01	C27
19102	2788	30.0	0.0	2.0	70254.0	2.0	2.0	200420.0	2.0	0.0	2020-11-01	C27
19103	2788	30.0	0.0	2.0	70254.0	2.0	2.0	411480.0	2.0	0.0	2020-12-01	C27

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
func={
    'Reportingdate':'count',
    'Dateofjoining':'first',
    'Target':'last',
    'Age': 'max',
    'Education_Level': 'max',
    'City': 'last',
    'Income':'last',
    'Joining Designation':'last',
    'Grade':'last',
    'Total Business Value':'sum',
    'Quarterly Rating':'last',
    'Driver_ID':'first',
    'Gender':'last',
    'LastWorkingDate':'last'

}

df1=df.groupby(['Driver_ID']).agg(func)
df1
```

Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quarter Rati
1	3	2018-12-24	1.0	28.0	2.0	C23	57387.0	1.0	1.0	1715580.0	
2	2	2020-11-06	0.0	31.0	2.0	C7	67016.0	2.0	2.0	0.0	
4	5	2019-12-07	1.0	43.0	2.0	C13	65603.0	2.0	2.0	350000.0	
5	3	2019-01-09	1.0	29.0	0.0	C9	46368.0	1.0	1.0	120360.0	
6	5	2020-07-31	0.0	31.0	1.0	C11	78728.0	3.0	3.0	1265000.0	
...	...	...	...	...	...	...	...	...	...	...	
2784	24	2015-10-15	0.0	34.0	0.0	C24	82815.0	2.0	3.0	21748820.0	
2785	3	2020-08-28	1.0	34.0	0.0	C9	12105.0	1.0	1.0	0.0	
2786	9	2018-07-31	1.0	45.0	0.0	C19	35370.0	2.0	2.0	2815090.0	
2787	6	2018-07-21	1.0	28.0	2.0	C20	69498.0	1.0	1.0	977830.0	
2788	7	2020-06-08	0.0	30.0	2.0	C27	70254.0	2.0	2.0	2298240.0	

```
df1.drop(columns=['Driver_ID'],inplace=True)
df1
```

Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quarter Rati
1	3	2018-12-24	1.0	28.0	2.0	C23	57387.0	1.0	1.0	1715580.0	
2	2	2020-11-06	0.0	31.0	2.0	C7	67016.0	2.0	2.0	0.0	
4	5	2019-12-07	1.0	43.0	2.0	C13	65603.0	2.0	2.0	350000.0	
5	3	2019-01-09	1.0	29.0	0.0	C9	46368.0	1.0	1.0	120360.0	
6	5	2020-07-31	0.0	31.0	1.0	C11	78728.0	3.0	3.0	1265000.0	
...	...	...	...	...	...	...	...	...	...	...	
2784	24	2015-10-15	0.0	34.0	0.0	C24	82815.0	2.0	3.0	21748820.0	
2785	3	2020-08-28	1.0	34.0	0.0	C9	12105.0	1.0	1.0	0.0	
2786	9	2018-07-31	1.0	45.0	0.0	C19	35370.0	2.0	2.0	2815090.0	
2787	6	2018-07-21	1.0	28.0	2.0	C20	69498.0	1.0	1.0	977830.0	
2788	7	2020-06-08	0.0	30.0	2.0	C27	70254.0	2.0	2.0	2298240.0	

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
df1.reset_index(inplace=True)
df1
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Qu
0	1	3	2018-12-24	1.0	28.0	2.0	C23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	C7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	C13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	C9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	C11	78728.0	3.0	3.0	1265000.0	
...	...	...	...	...	...	...	...	...	...	...	...	
2376	2784	24	2015-10-15	0.0	34.0	0.0	C24	82815.0	2.0	3.0	21748820.0	
2377	2785	3	2020-08-28	1.0	34.0	0.0	C9	12105.0	1.0	1.0	0.0	
2378	2786	9	2018-07-31	1.0	45.0	0.0	C19	35370.0	2.0	2.0	2815090.0	
2379	2787	6	2018-07-21	1.0	28.0	2.0	C20	69498.0	1.0	1.0	977830.0	
2380	2788	7	2020-06-08	0.0	30.0	2.0	C27	70254.0	2.0	2.0	2298240.0	

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
# Creating new column to check if the drivers got any rise in rating

QR_First= df.groupby('Driver_ID')['Quarterly Rating'].first().reset_index()
QR_First
```

	Driver_ID	Quarterly Rating
0	1	2.0
1	2	1.0
2	4	1.0
3	5	1.0
4	6	1.0
...	...	...
2376	2784	3.0
2377	2785	1.0
2378	2786	2.0
2379	2787	2.0
2380	2788	1.0

2381 rows × 2 columns

Next steps: Generate code with QR_First View recommended plots New interactive sheet

```
QR_Last= df.groupby('Driver_ID')['Quarterly Rating'].last().reset_index()
QR_Last
```

	Driver_ID	Quarterly Rating
0	1	2.0
1	2	1.0
2	4	1.0
3	5	1.0
4	6	2.0
...	...	...
2376	2784	4.0
2377	2785	1.0
2378	2786	1.0
2379	2787	1.0
2380	2788	2.0

2381 rows × 2 columns

Next steps: Generate code with QR_Last View recommended plots New interactive sheet

```
df1['Rating_Rise']=np.where(QR_Last['Quarterly Rating']>QR_First['Quarterly Rating'],1,0)
df1
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Qu
0	1	3	2018-12-24	1.0	28.0	2.0	C23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	C7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	C13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	C9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	C11	78728.0	3.0	3.0	1265000.0	
...	...	...	...	...	...	...	...	...	...	...	...	
2376	2784	24	2015-10-15	0.0	34.0	0.0	C24	82815.0	2.0	3.0	21748820.0	
2377	2785	3	2020-08-28	1.0	34.0	0.0	C9	12105.0	1.0	1.0	0.0	
2378	2786	9	2018-07-31	1.0	45.0	0.0	C19	35370.0	2.0	2.0	2815090.0	
2379	2787	6	2018-07-21	1.0	28.0	2.0	C20	69498.0	1.0	1.0	977830.0	
2380	2788	7	2020-06-08	0.0	30.0	2.0	C27	70254.0	2.0	2.0	2298240.0	

2381 rows × 15 columns

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
df1['Rating_Rise'].value_counts()
```

Rating_Rise	count
0	2023
1	358

dtype: int64

```
# Checking if there is income rise for the drivers

Inc_First= df.groupby('Driver_ID')['Income'].first().reset_index()
Inc_First
```

	Driver_ID	Income
0	1	57387.0
1	2	67016.0
2	4	65603.0
3	5	46368.0
4	6	78728.0
...	...	...
2376	2784	82815.0
2377	2785	12105.0
2378	2786	35370.0
2379	2787	69498.0
2380	2788	70254.0

2381 rows × 2 columns

Next steps: Generate code with Inc_First | View recommended plots | New interactive sheet

```
Inc_Last= df.groupby('Driver_ID')['Income'].last().reset_index()
Inc_Last
```

	Driver_ID	Income
0	1	57387.0
1	2	67016.0
2	4	65603.0
3	5	46368.0
4	6	78728.0
...	...	...
2376	2784	82815.0
2377	2785	12105.0
2378	2786	35370.0
2379	2787	69498.0
2380	2788	70254.0

2381 rows × 2 columns

Next steps: Generate code with Inc_Last | View recommended plots | New interactive sheet

```
df1['Income_Rise']=np.where(Inc_Last['Income']>Inc_First['Income'],1,0)
df1
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Qu
0	1	3	2018-12-24	1.0	28.0	2.0	C23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	C7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	C13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	C9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	C11	78728.0	3.0	3.0	1265000.0	
...	...	...	...	...	...	...	...	...	...	...	...	
2376	2784	24	2015-10-15	0.0	34.0	0.0	C24	82815.0	2.0	3.0	21748820.0	
2377	2785	3	2020-08-28	1.0	34.0	0.0	C9	12105.0	1.0	1.0	0.0	
2378	2786	9	2018-07-31	1.0	45.0	0.0	C19	35370.0	2.0	2.0	2815090.0	
2379	2787	6	2018-07-21	1.0	28.0	2.0	C20	69498.0	1.0	1.0	977830.0	
2380	2788	7	2020-06-08	0.0	30.0	2.0	C27	70254.0	2.0	2.0	2298240.0	

2381 rows × 16 columns

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
df1['Income_Rise'].value_counts()
```

Income_Rise	count
0	2338
1	43

dtype: int64

```
#checking if there is any grade change

Grade_First= df.groupby('Driver_ID')['Grade'].first().reset_index()
Grade_First
```

	Driver_ID	Grade
0	1	1.0
1	2	2.0
2	4	2.0
3	5	1.0
4	6	3.0
...	...	...
2376	2784	3.0
2377	2785	1.0
2378	2786	2.0
2379	2787	1.0
2380	2788	2.0

2381 rows × 2 columns

Next steps: Generate code with Grade_First View recommended plots New interactive sheet

```
Grade_Last= df.groupby('Driver_ID')['Grade'].last().reset_index()
Grade_Last
```

	Driver_ID	Grade
0	1	1.0
1	2	2.0
2	4	2.0
3	5	1.0
4	6	3.0
...	...	...
2376	2784	3.0
2377	2785	1.0
2378	2786	2.0
2379	2787	1.0
2380	2788	2.0

2381 rows × 2 columns

Next steps: Generate code with Grade_Last View recommended plots New interactive sheet

```
df1['Grade_Rise']=np.where(Grade_Last['Grade']>Grade_First['Grade'],1,0)
df1
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Qu
0	1	3	2018-12-24	1.0	28.0	2.0	C23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	C7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	C13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	C9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	C11	78728.0	3.0	3.0	1265000.0	
...	...	...	...	...	...	...	...	...	...	...	...	
2376	2784	24	2015-10-15	0.0	34.0	0.0	C24	82815.0	2.0	3.0	21748820.0	
2377	2785	3	2020-08-28	1.0	34.0	0.0	C9	12105.0	1.0	1.0	0.0	
2378	2786	9	2018-07-31	1.0	45.0	0.0	C19	35370.0	2.0	2.0	2815090.0	
2379	2787	6	2018-07-21	1.0	28.0	2.0	C20	69498.0	1.0	1.0	977830.0	
2380	2788	7	2020-06-08	0.0	30.0	2.0	C27	70254.0	2.0	2.0	2298240.0	

2381 rows × 17 columns

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
df1['Grade_Rise'].value_counts()
```

Grade_Rise	count
0	2338
1	43

dtype: int64

```
df1['City']= df1['City'].astype(str).str.strip().str[1:]
df1.head()
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quart Ra
0	1	3	2018-12-24	1.0	28.0	2.0	23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	11	78728.0	3.0	3.0	1265000.0	

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
df1['month_of_joining']=pd.to_datetime(df1['Dateofjoining']).dt.month
df1['year_of_joining']=pd.to_datetime(df1['Dateofjoining']).dt.year
df1.head()
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quart Ra
0	1	3	2018-12-24	1.0	28.0	2.0	23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	11	78728.0	3.0	3.0	1265000.0	

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
# Checking duration of drivers in OLA

df1['Driver_tenure']=((df1['LastWorkingDate']-df1['Dateofjoining']).dt.days/30).round(1)

df1.head()
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quart Ra
0	1	3	2018-12-24	1.0	28.0	2.0	23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	11	78728.0	3.0	3.0	1265000.0	

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
df1['Driver_tenure'].fillna(((df1['Dateofjoining'].max()-df1['Dateofjoining']).dt.days/30.44).round(1),inplace=True)
df1.head()
```

	Driver_ID	Reportingdate	Dateofjoining	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quart Ra
0	1	3	2018-12-24	1.0	28.0	2.0	23	57387.0	1.0	1.0	1715580.0	
1	2	2	2020-11-06	0.0	31.0	2.0	7	67016.0	2.0	2.0	0.0	
2	4	5	2019-12-07	1.0	43.0	2.0	13	65603.0	2.0	2.0	350000.0	
3	5	3	2019-01-09	1.0	29.0	0.0	9	46368.0	1.0	1.0	120360.0	
4	6	5	2020-07-31	0.0	31.0	1.0	11	78728.0	3.0	3.0	1265000.0	

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
df1.rename(columns={'Reportingdate':'Reportings'},inplace=True)
df1.columns
```

```
Index(['Driver_ID', 'Reportings', 'Dateofjoining', 'Target', 'Age',
       'Education_Level', 'City', 'Income', 'Joining Designation', 'Grade',
       'Total Business Value', 'Quarterly Rating', 'Gender', 'LastWorkingDate',
       'Rating_Rise', 'Income_Rise', 'Grade_Rise', 'month_of_joining',
       'year_of_joining', 'Driver_tenure'],
      dtype='object')
```

```
df1.drop(columns=['Dateofjoining','LastWorkingDate'],inplace=True)
df1.columns
```

```
Index(['Driver_ID', 'Reportings', 'Target', 'Age', 'Education_Level', 'City',
       'Income', 'Joining Designation', 'Grade', 'Total Business Value',
       'Quarterly Rating', 'Gender', 'Rating_Rise', 'Income_Rise',
       'Grade_Rise', 'month_of_joining', 'year_of_joining', 'Driver_tenure'],
      dtype='object')
```

```
#Changing the data types to integer data types


df1['Age']=df1['Age'].astype('int64')
df1['Gender']=df1['Gender'].astype('int64')
df1['City']=df1['City'].astype('int64')
df1['Quarterly Rating']=df1['Quarterly Rating'].astype('int64')
df1['Target']=df1['Target'].astype('int64')
df1['Joining Designation']=df1['Joining Designation'].astype('int64')
df1['Grade']=df1['Grade'].astype('int64')
df1['Education_Level'].value_counts()
df1['Education_Level']=df1['Education_Level'].astype('int64')
```

```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2381 entries, 0 to 2380
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Driver_ID             2381 non-null   int64  
 1   Reportings            2381 non-null   int64  
 2   Target                2381 non-null   int64  
 3   Age                   2381 non-null   int64  
 4   Education_Level       2381 non-null   int64  
 5   City                  2381 non-null   int64  
 6   Income                2381 non-null   float64
 7   Joining Designation   2381 non-null   int64  
 8   Grade                 2381 non-null   int64  
 9   Total Business Value  2381 non-null   float64
 10  Quarterly Rating      2381 non-null   int64  
 11  Gender                2381 non-null   int64  
 12  Rating_Rise           2381 non-null   int64  
 13  Income_Rise           2381 non-null   int64  
 14  Grade_Rise            2381 non-null   int64  
 15  month_of_joining      2381 non-null   int32  
 16  year_of_joining       2381 non-null   int32  
 17  Driver_tenure         2381 non-null   float64
dtypes: float64(3), int32(2), int64(13)
memory usage: 316.4 KB
```

```
ola=df1
ola
```

	Driver_ID	Reportings	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Gender	R
0	1	3	1	28	2	23	57387.0	1	1	1715580.0	2	0	
1	2	2	0	31	2	7	67016.0	2	2	0.0	1	0	
2	4	5	1	43	2	13	65603.0	2	2	350000.0	1	0	
3	5	3	1	29	0	9	46368.0	1	1	120360.0	1	0	
4	6	5	0	31	1	11	78728.0	3	3	1265000.0	2	1	
...	...	...	...	...	...	...	...	...	...	...	...	...	
2376	2784	24	0	34	0	24	82815.0	2	3	21748820.0	4	0	
2377	2785	3	1	34	0	9	12105.0	1	1	0.0	1	1	
2378	2786	9	1	45	0	19	35370.0	2	2	2815090.0	1	0	
2379	2787	6	1	28	2	20	69498.0	1	1	977830.0	1	1	
2380	2788	7	0	30	2	27	70254.0	2	2	2298240.0	2	0	

2381 rows × 18 columns

Next steps: Generate code with df1 View recommended plots New interactive sheet

```
ola.isna().sum()
```

	0
Driver_ID	0
Reportings	0
Target	0
Age	0
Education_Level	0
City	0
Income	0
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0
Gender	0
Rating_Rise	0
Income_Rise	0
Grade_Rise	0
month_of_joining	0
year_of_joining	0
Driver_tenure	0

dtype: int64

```
ola.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Driver_ID	2381.0	1.397559e+03	8.061616e+02	1.0	695.0	1400.0	2100.0	2788.0
Reportings	2381.0	8.023520e+00	6.783590e+00	1.0	3.0	5.0	10.0	24.0
Target	2381.0	6.787064e-01	4.670713e-01	0.0	0.0	1.0	1.0	1.0
Age	2381.0	3.374171e+01	5.944850e+00	21.0	30.0	33.0	37.0	58.0
Education_Level	2381.0	1.007560e+00	8.162900e-01	0.0	0.0	1.0	2.0	2.0
City	2381.0	1.533557e+01	8.371843e+00	1.0	8.0	15.0	22.0	29.0
Income	2381.0	5.933416e+04	2.838367e+04	10747.0	39104.0	55315.0	75986.0	188418.0
Joining Designation	2381.0	1.820244e+00	8.414334e-01	1.0	1.0	2.0	2.0	5.0
Grade	2381.0	2.096598e+00	9.415218e-01	1.0	1.0	2.0	3.0	5.0
Total Business Value	2381.0	4.586742e+06	9.127115e+06	-1385530.0	0.0	817680.0	4173650.0	95331060.0
Quarterly Rating	2381.0	1.427971e+00	8.098389e-01	1.0	1.0	1.0	2.0	4.0
Gender	2381.0	4.094918e-01	4.918433e-01	0.0	0.0	0.0	1.0	1.0
Rating_Rise	2381.0	1.503570e-01	3.574961e-01	0.0	0.0	0.0	0.0	1.0
Income_Rise	2381.0	1.805964e-02	1.331951e-01	0.0	0.0	0.0	0.0	1.0
Grade_Rise	2381.0	1.805964e-02	1.331951e-01	0.0	0.0	0.0	0.0	1.0
month_of_joining	2381.0	7.357413e+00	3.143143e+00	1.0	5.0	7.0	10.0	12.0
year_of_joining	2381.0	2.018536e+03	1.609597e+00	2013.0	2018.0	2019.0	2020.0	2020.0
Driver_tenure	2381.0	1.444427e+01	1.874392e+01	0.0	3.3	6.4	15.9	92.9

- This dataset has details of 2381 Ola Drivers collected from January 2013 to December 2020. (i.e over a span of 7 years)
- Minimam age of the drivers is 21 and maximum age 58 years
- Majority of the drivers are from the City 22.
- 75% of the drivers average income is 75986.0
- 50% of the drivers who left the organisation, had stayed around only 6.4 months.

```
ola['Target'].value_counts()
```

Target	count
1	1616
0	765

dtype: int64

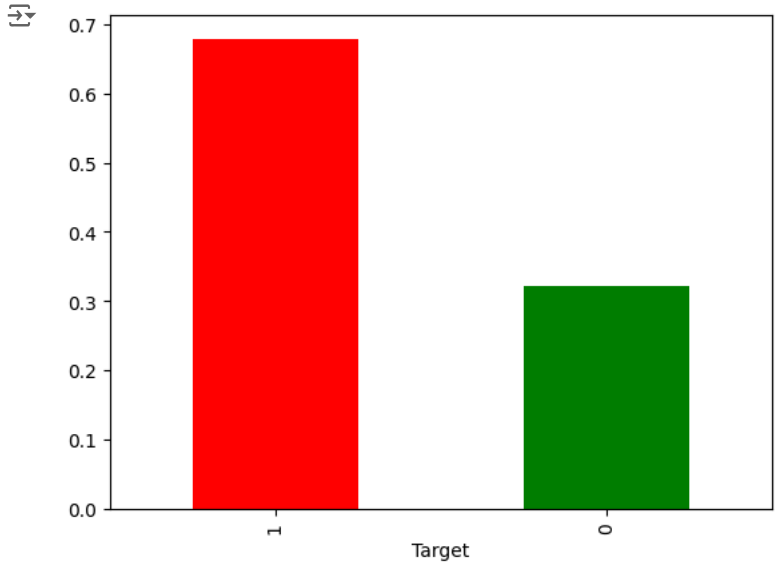
Out of 2381 drivers, 1616 people left the organization.

```
ola['Target'].value_counts(normalize=True)*100
```

Target	proportion
1	67.870643
0	32.129357

dtype: float64

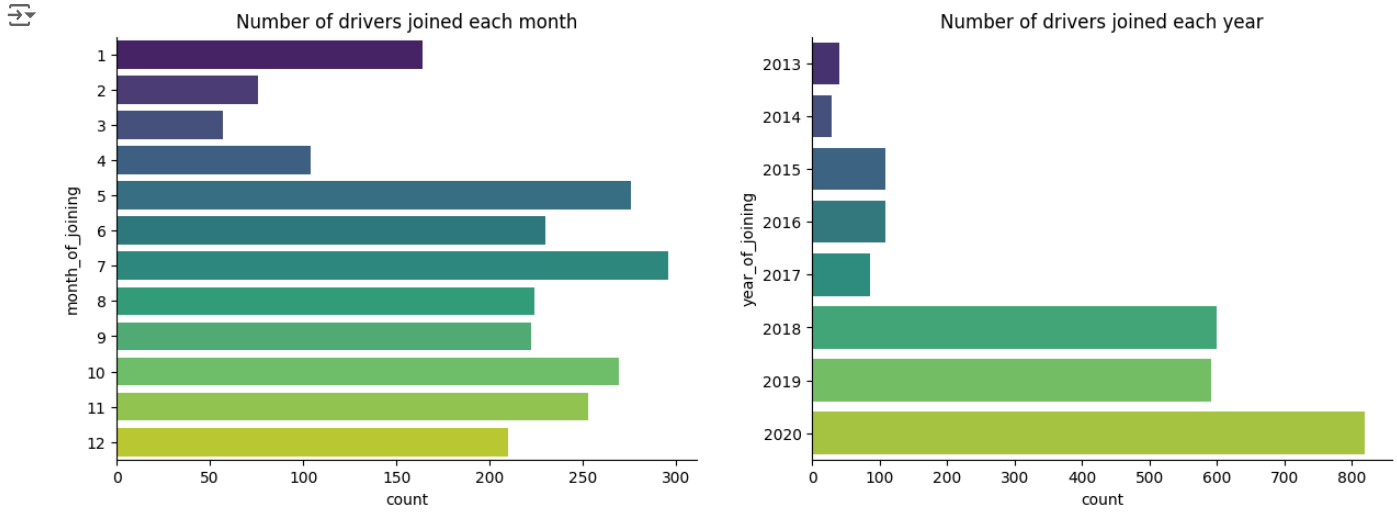
```
ola['Target'].value_counts(normalize=True).plot(kind='bar',color=['red','green'])
plt.show()
```



Around 68% of the drivers left Ola .

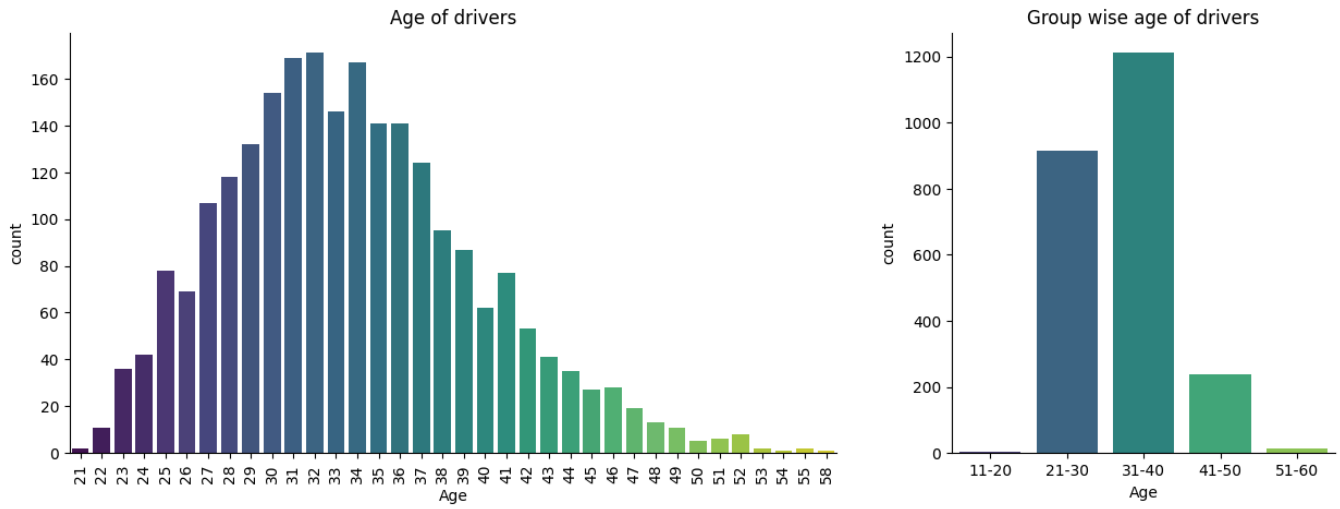
Univariate Analysis

```
# Number of drivers joined

fig, ax = plt.subplots(1, 2, figsize=(15, 5))

# Sort the data based on frequency for month_of_joining
month_order = ola['month_of_joining'].value_counts().index

# Sort the data based on frequency for year_of_joining
year_order = ola['year_of_joining'].value_counts().index

# Plot for drivers joined each month
sns.countplot(data=ola, y='month_of_joining', ax=ax[0],  palette='viridis')
ax[0].set_title('Number of drivers joined each month')

# Plot for drivers joined each year
sns.countplot(data=ola, y='year_of_joining', ax=ax[1], palette='viridis')
#sns.countplot(data=ola, y='year_of_joining', ax=ax[1],order= year_order, palette='viridis')
ax[1].set_title('Number of drivers joined each year')

# Show the plot
sns.despine()
plt.show()
```



Max number of drivers joined in July , followed by May and October months. There is a tremendous growth in drivers joining OLA after 2017 and reached maximum by 2020.

```
fig,ax= plt.subplots(1,2,figsize=(15,5),gridspec_kw={'width_ratios': [2, 1]})

# Plot 1: Age of Drivers
sns.countplot(x=ola['Age'],ax=ax[0], palette='viridis',width=0.8)
ax[0].set_title('Age of drivers')
ax[0].tick_params(axis='x', rotation=90)

# Plot 2: Group wise age of drivers
age_groups= pd.cut(ola['Age'],bins=[11,21,31,41,51,61], labels=['11-20','21-30','31-40','41-50','51-60'])
sns.countplot(x=age_groups,ax=ax[1], palette='viridis')
ax[1].set_title('Group wise age of drivers')

# Show the plot
sns.despine()
plt.show()
```



Maximum drivers are in the age group pf 21 to 40 years.

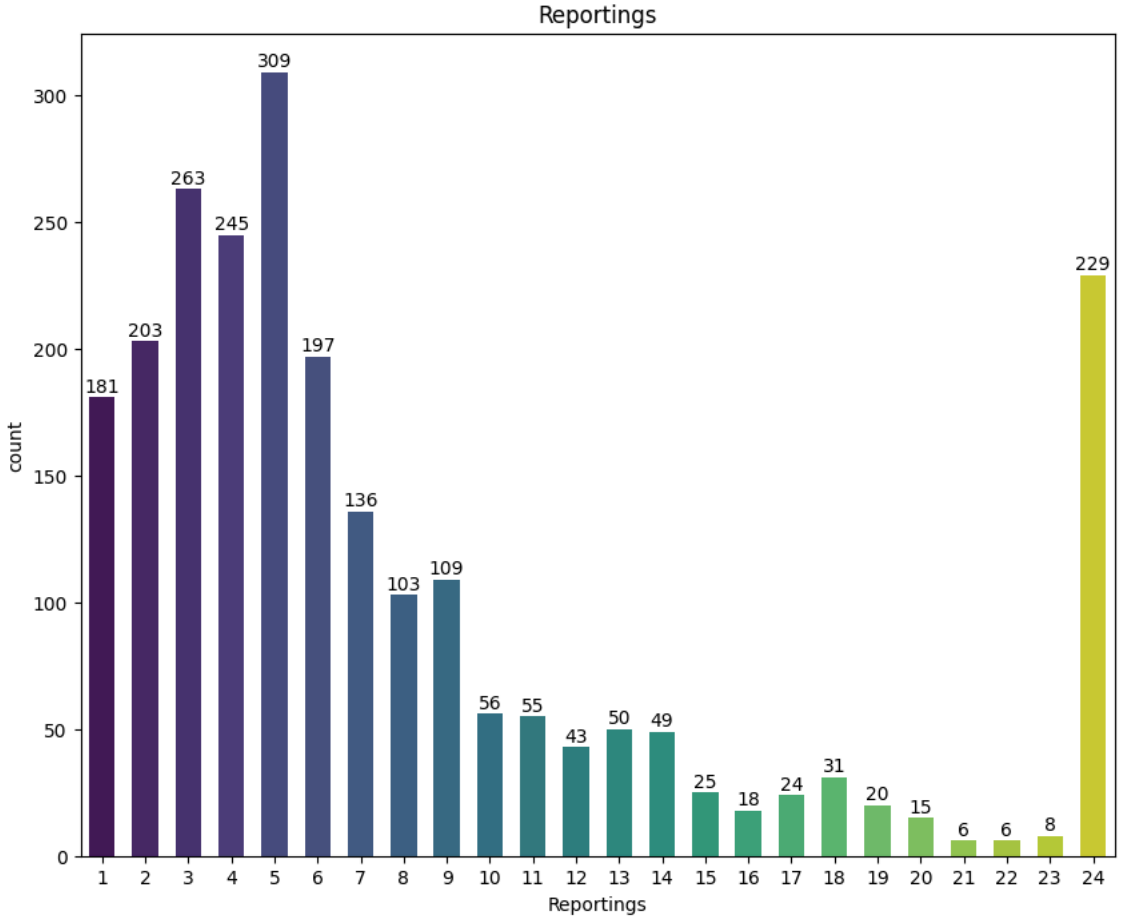
```
ola.head()
```

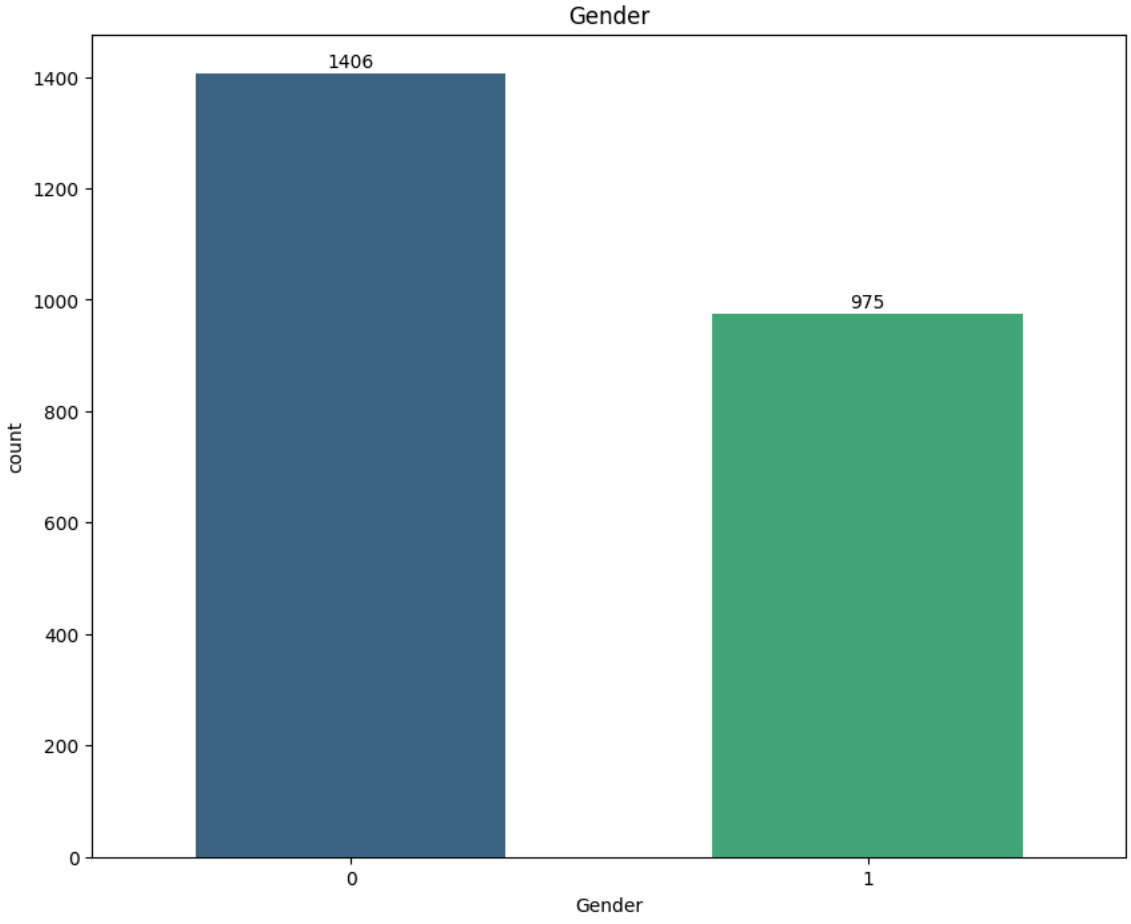
	Driver_ID	Reportings	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Gender	Ratir
0	1	3	1	28	2	23	57387.0	1	1	1715580.0	2	0	
1	2	2	0	31	2	7	67016.0	2	2	0.0	1	0	
2	4	5	1	43	2	13	65603.0	2	2	350000.0	1	0	
3	5	3	1	29	0	9	46368.0	1	1	120360.0	1	0	
4	6	5	0	31	1	11	78728.0	3	3	1265000.0	2	1	

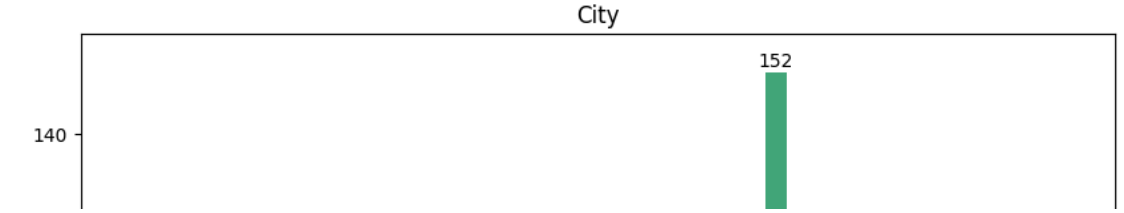
Next steps: Generate code with ola | View recommended plots | New interactive sheet

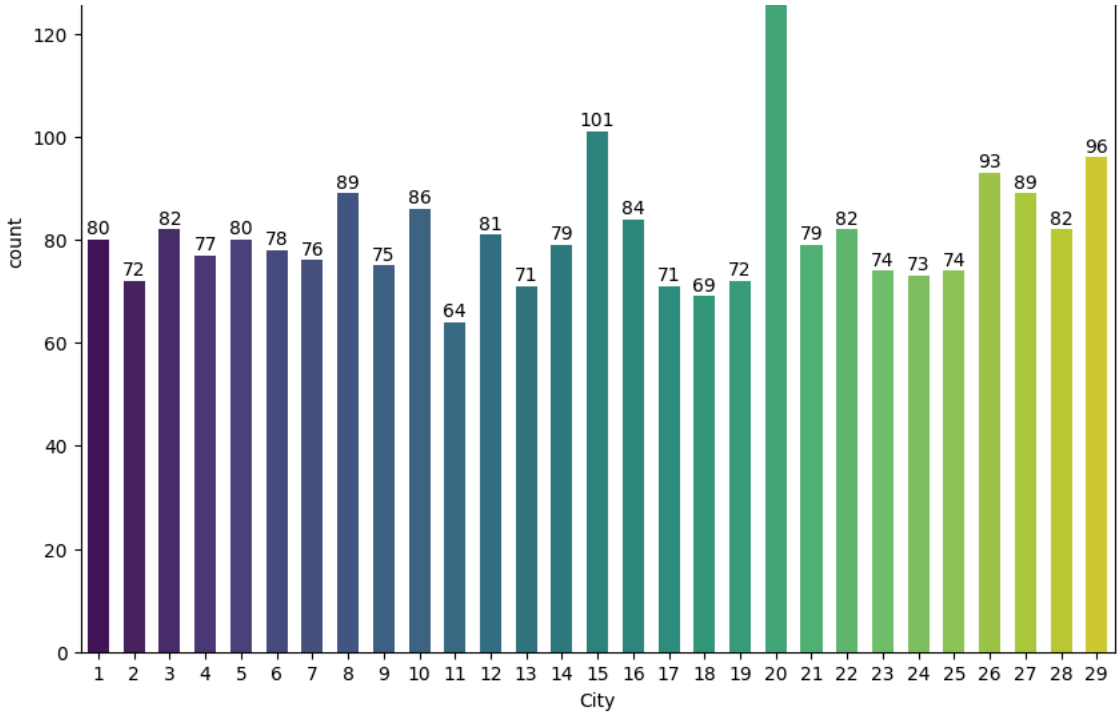
```
# Plotting Categorical columns

cat_cols=['Reportings','Gender','City','Education_Level','Joining Designation','Grade','Quarterly Rating','Rating_Rise','Inc

for i in range(len(cat_cols)):
  plt.figure(figsize=(10, 8))
  ax=sns.countplot(x=ola[cat_cols[i]],palette='viridis',width=0.6)

  # Add value counts above the bars
  for p in ax.patches:
      ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2., p.get_height()),
                  ha='center', va='baseline', fontsize=10, color='black', xytext=(0, 3),
                  textcoords='offset points')
```

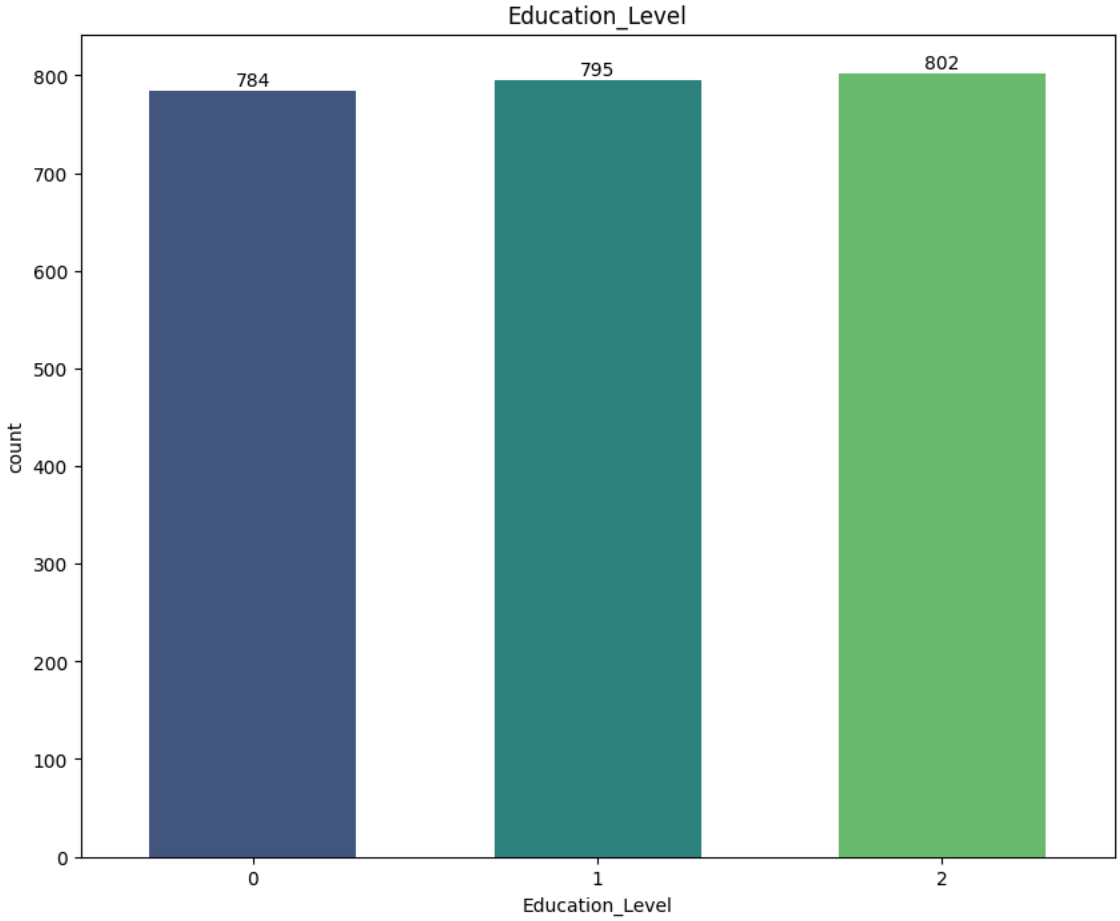
```
    plt.title(cat_cols[i])
    plt.show()
```

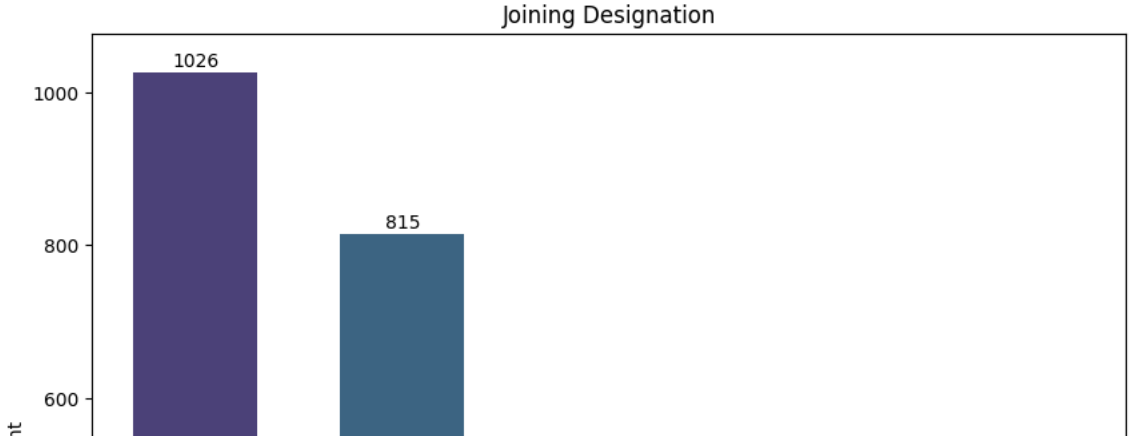

Reportings
count
Reportings
181
203
263
245
309
197
136
103
109
56
55
43
50
49
25
18
24
31
20
15
6
6
8
229

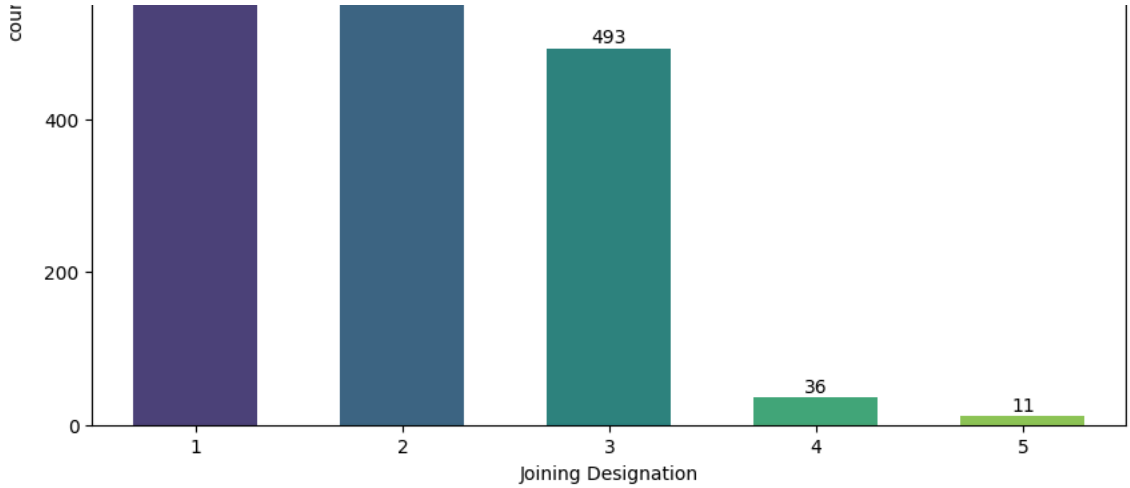

Gender
count
Gender
1406
975

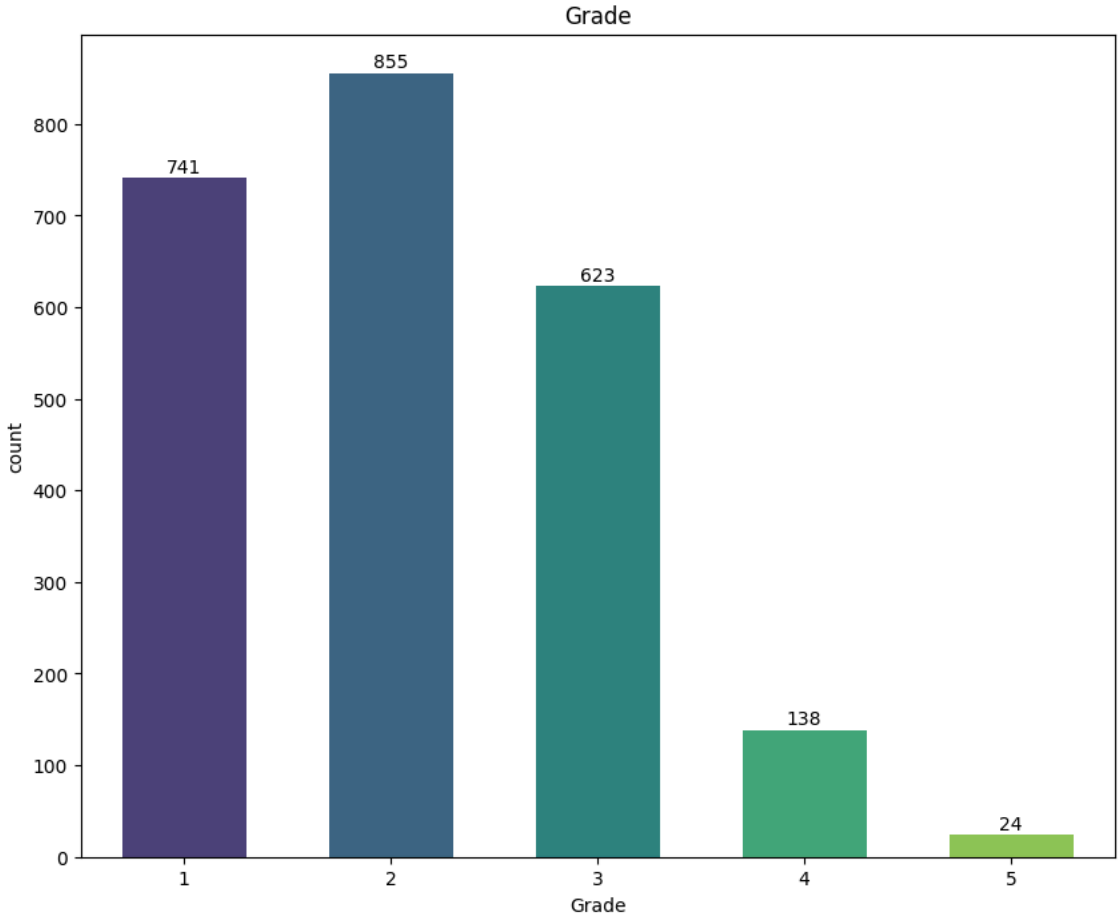

City
152

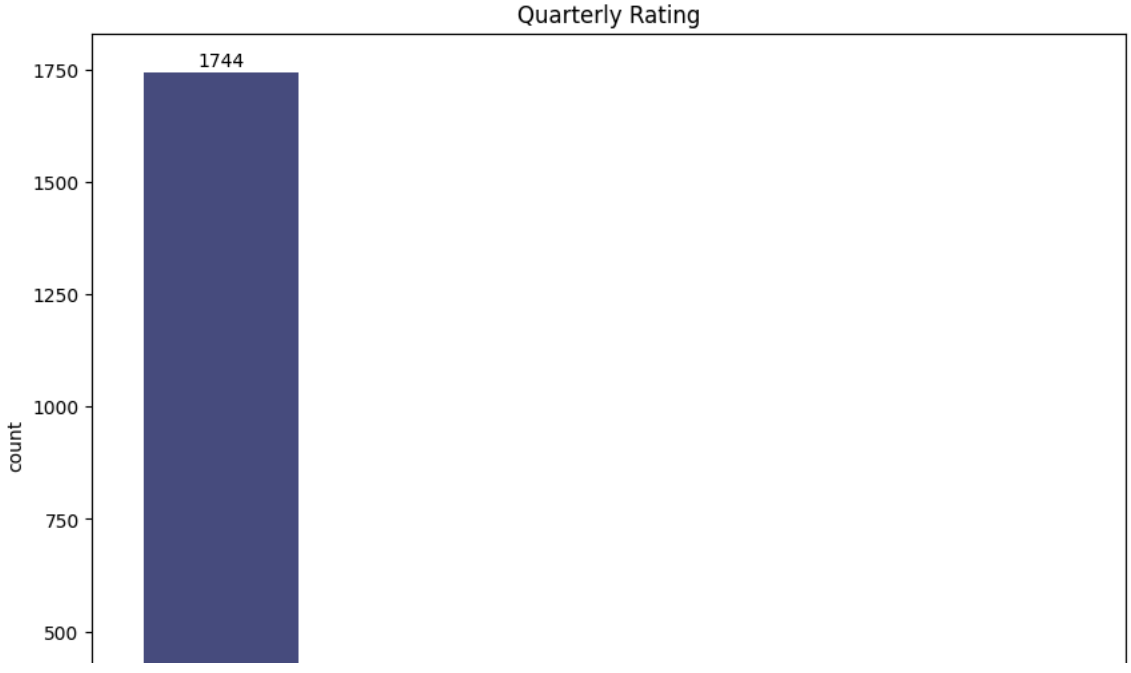

count
City
80 72 82 77 80 78 76 89 75 86 64 81 71 79 101 84 71 69 72 120 79 82 74 73 74 93 89 82 96
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29

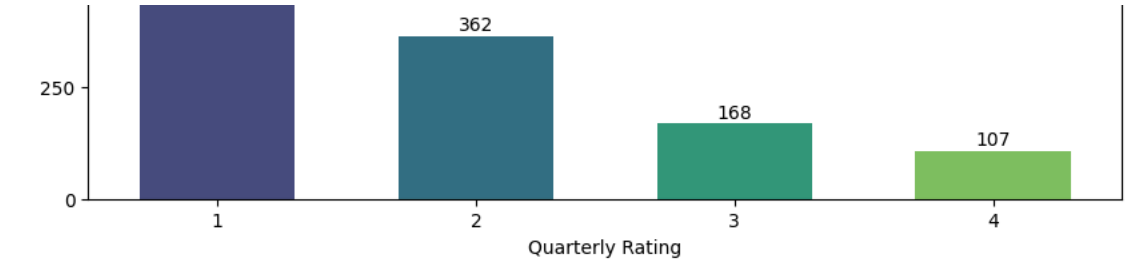

Education_Level
count
Education_Level
784 795 802
0 1 2

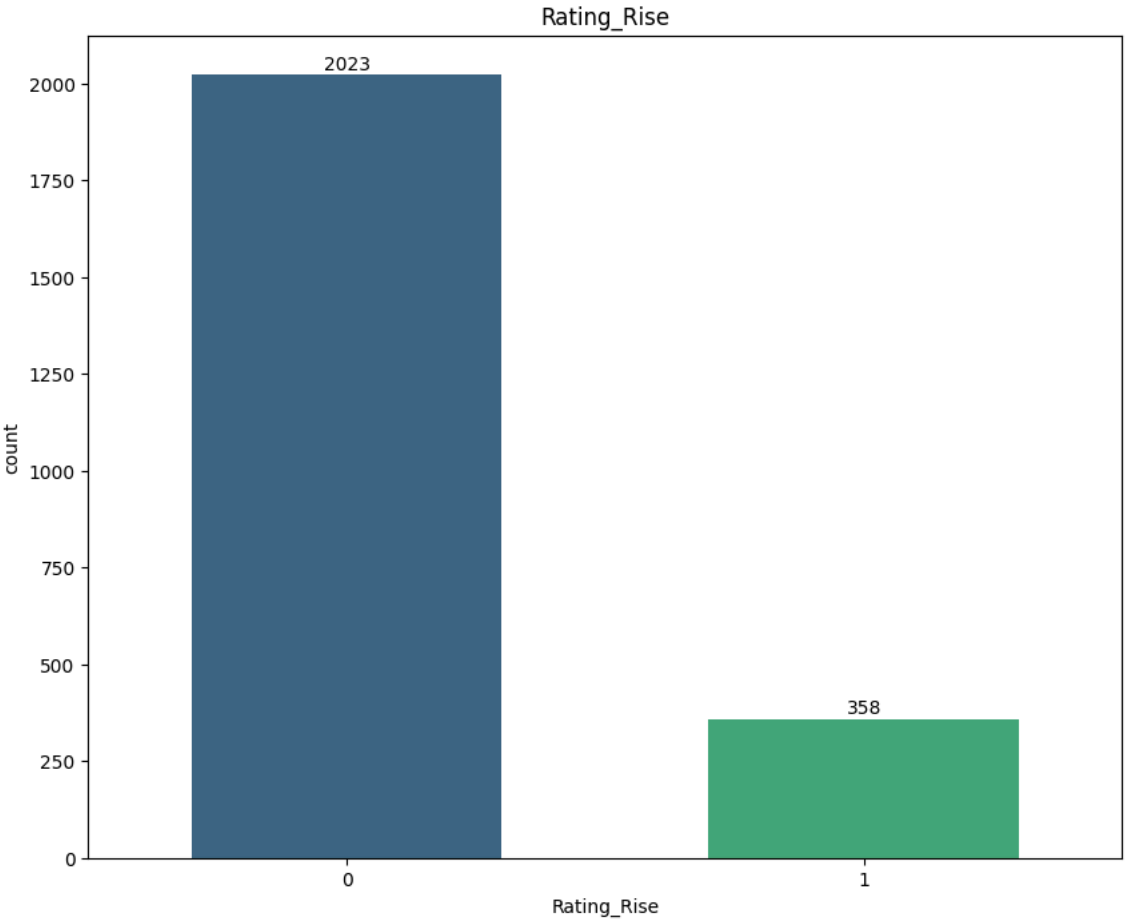

Joining Designation
1026 815

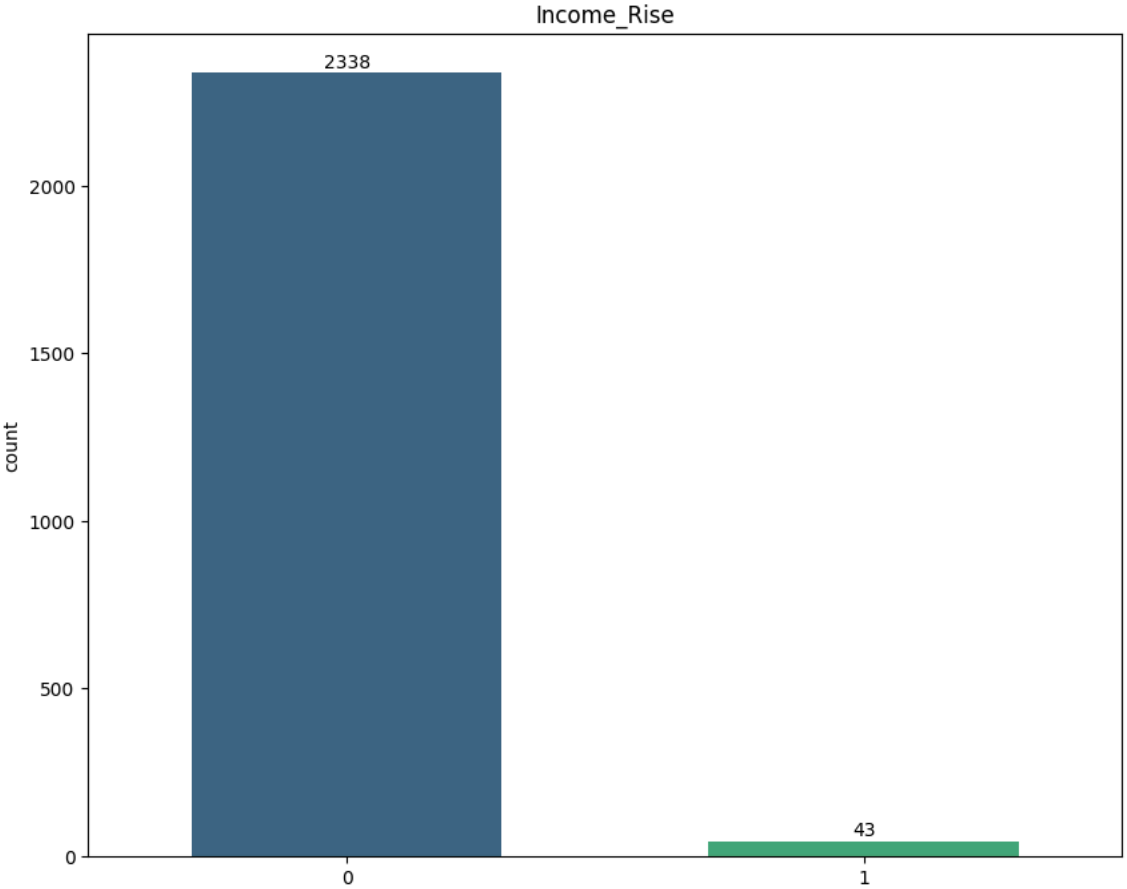

cour
400
200
0
493
36
11
1
2
3
4
5
Joining Designation

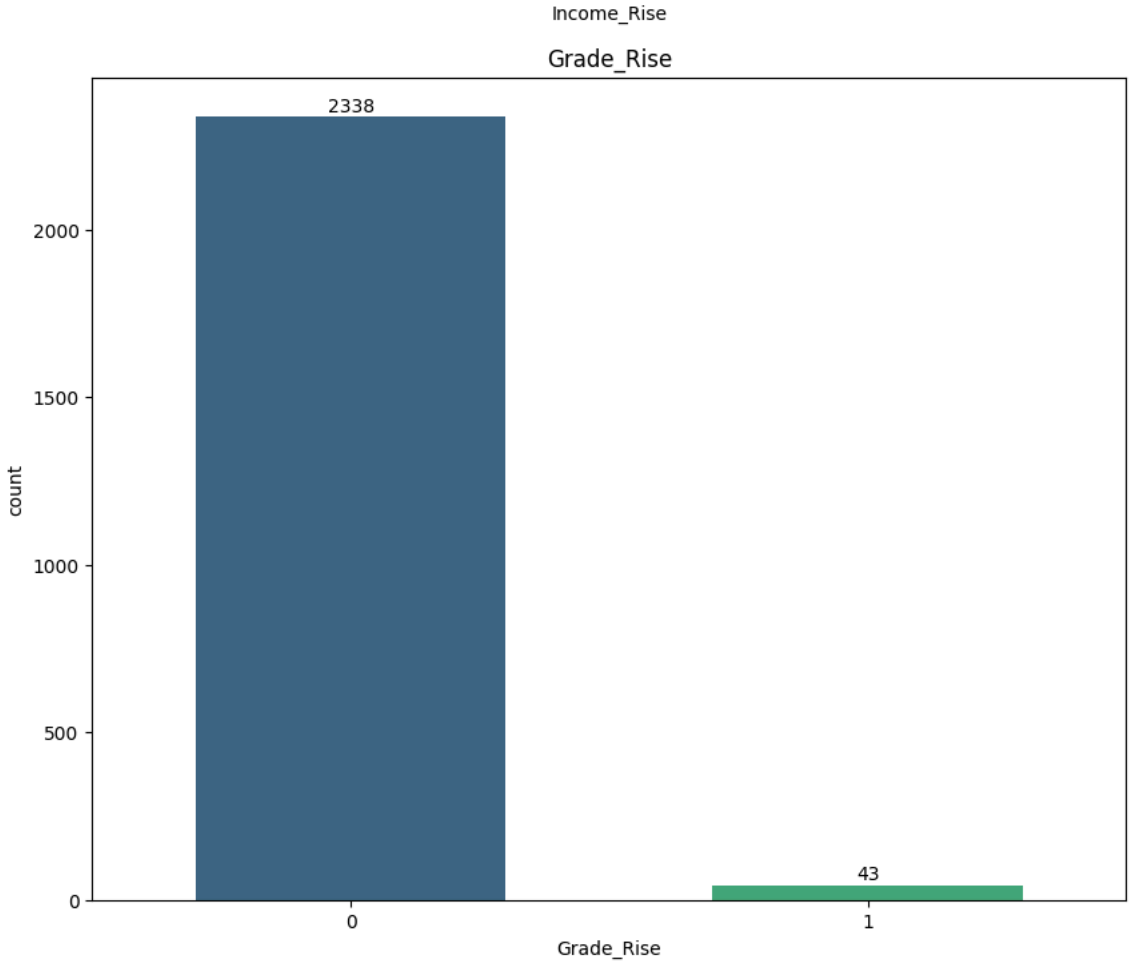

Grade
855
800
741
700
623
600
count
500
400
300
200
138
100
0
24
1
2
3
4
5
Grade


Quarterly Rating
1744
1750
1500
1250
1000
count
750
500


362
168
107
250
0
1
2
3
4
Quarterly Rating

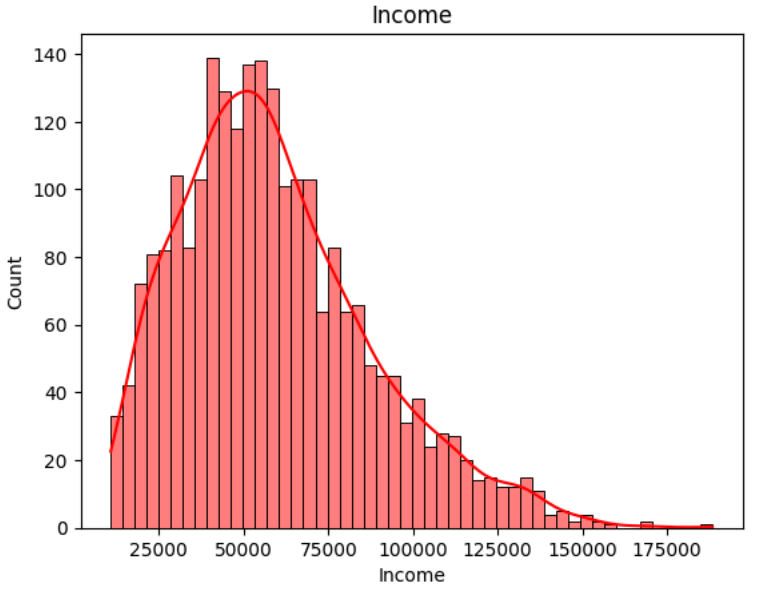

Rating_Rise
2023
358
2000
1750
1500
1250
1000
750
500
250
0
count
0
1
Rating_Rise

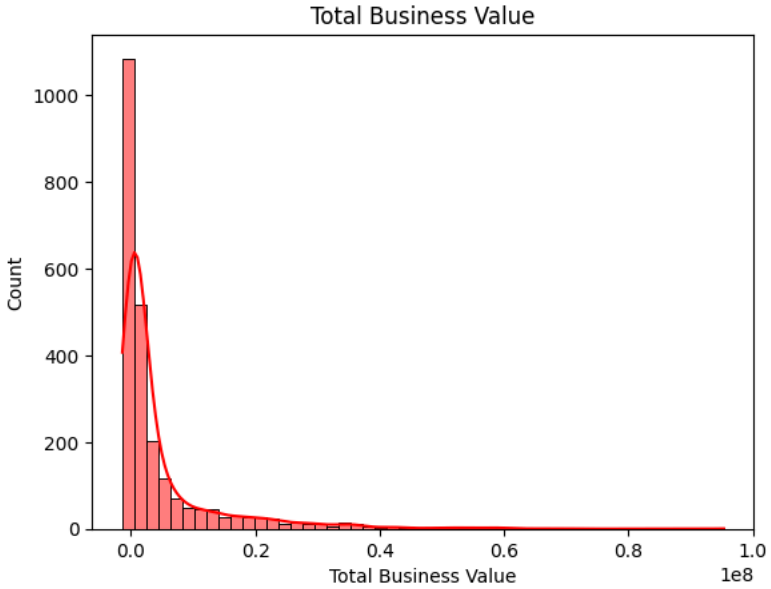

Income_Rise
2338
43
2000
1500
1000
500
0
count
0
1

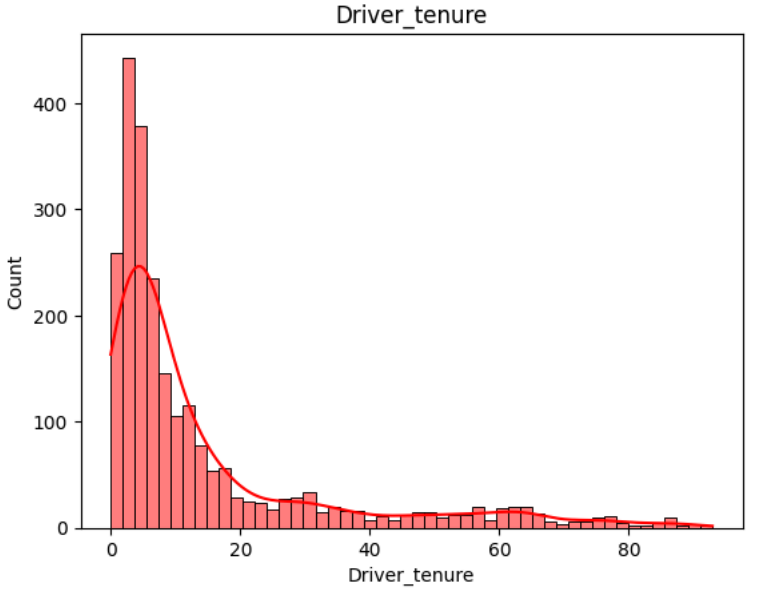

Income_Rise
Grade_Rise
2338
43
2000
1500
1000
500
0
count
0
1
Grade_Rise

- Out of 2381 Drivers, 1406 are Male and 975 are Female.
- Majority of the drivers are from cities C20, C15 and C29
- There are almost equal number of drivers from all educational levels .
- Very few drivers (43) got rise in income and grades.
- Only 358 drivers got rise in their quarterly ratings.

```
# Plotting Categorical columns

num_cols=['Income','Total Business Value','Driver_tenure']

for i in range(len(num_cols)):
  ax=sns.histplot(ola[num_cols[i]],bins=50,kde=True,color='red')
  ax.set_xlabel(num_cols[i])
  plt.title(num_cols[i])
  plt.show()
```







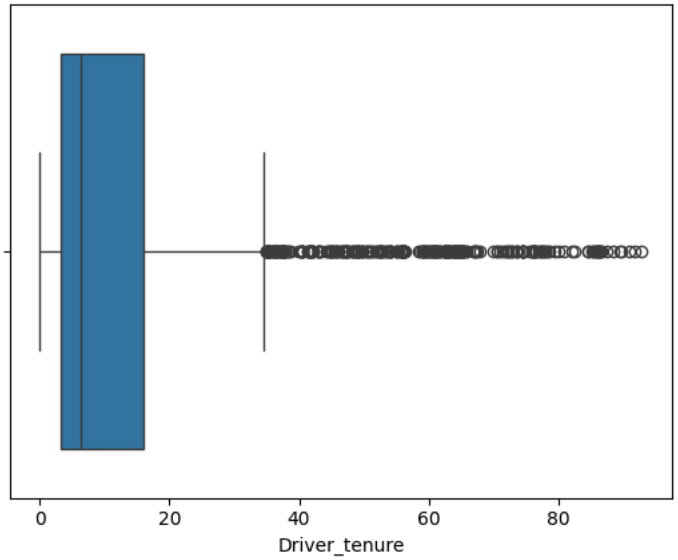

```
ola['Driver_tenure'].describe()
```

	Driver_tenure
count	2381.000000
mean	14.444267
std	18.743919
min	0.000000
25%	3.300000
50%	6.400000
75%	15.900000
max	92.900000

dtype: float64

```
sns.boxplot(x=ola['Driver_tenure'])
```

```
<Axes: xlabel='Driver_tenure'>
```




```
ola['Driver_tenure'].describe()
```

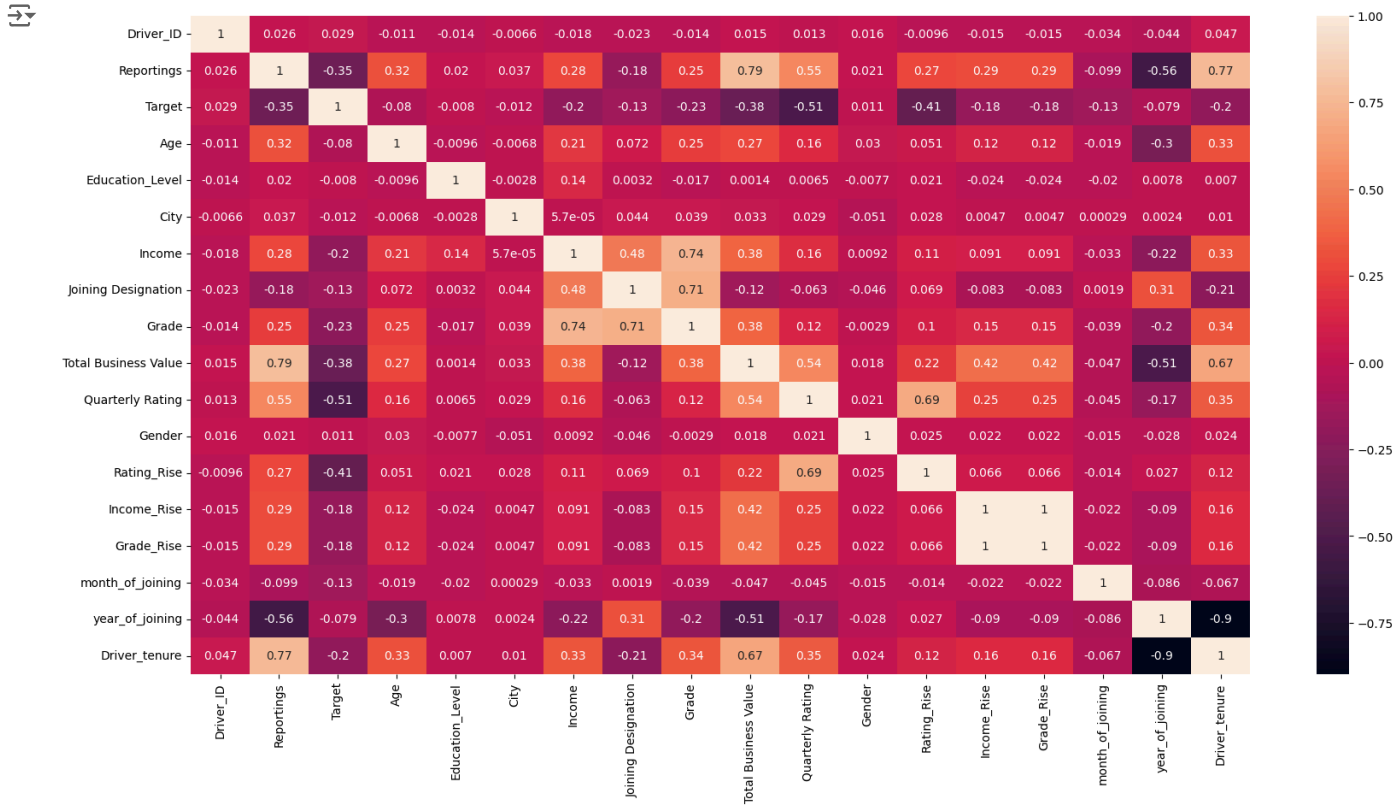
	Driver_tenure
count	2381.000000
mean	14.444267
std	18.743919
min	0.000000
25%	3.300000
50%	6.400000
75%	15.900000
max	92.900000

dtype: float64

- Out of 2381 Drivers , 75% of them left the organisation within 16 months .
- 50% of the drivers left OLA within 6.4 months of joining .

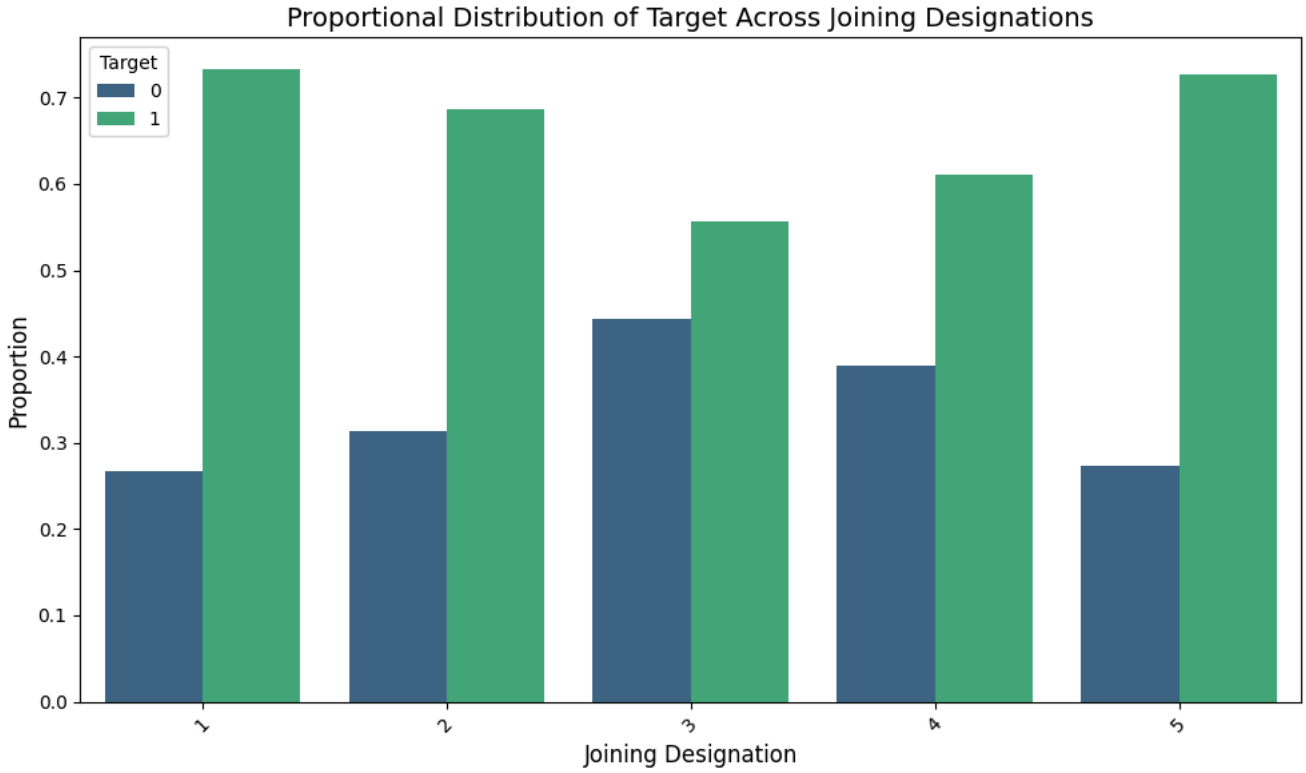
Bivariate Analysis

```
corr= ola.corr()
corr

plt.figure(figsize=(20,10))
sns.heatmap(corr,annot=True)
plt.show()
```




```
# Calculate normalized proportions for each Joining Designation
jde = pd.crosstab(ola['Joining Designation'], ola['Target'], normalize='index').reset_index()
print(jde)
# Melt the dataframe for easier plotting with seaborn
jde_melted = jde.melt(id_vars='Joining Designation', var_name='Target', value_name='Proportion')

# Plot using seaborn
plt.figure(figsize=(10,6))
sns.barplot(data=jde_melted, x='Joining Designation', y='Proportion', hue='Target', palette='viridis')

# Add labels and title
plt.title("Proportional Distribution of Target Across Joining Designations", fontsize=14)
plt.xlabel("Joining Designation", fontsize=12)
plt.ylabel("Proportion", fontsize=12)
plt.xticks(rotation=45)
plt.legend(title="Target")
plt.tight_layout()

plt.show()
```

```
Target  Joining Designation         0         1
0                       1  0.267057  0.732943
1                       2  0.312883  0.687117
2                       3  0.444219  0.555781
3                       4  0.388889  0.611111
4                       5  0.272727  0.727273
```



`jde_melted`

	Joining Designation	Target	Proportion
0	1	0	0.267057
1	2	0	0.312883
2	3	0	0.444219
3	4	0	0.388889
4	5	0	0.272727
5	1	1	0.732943
6	2	1	0.687117
7	3	1	0.555781
8	4	1	0.611111
9	5	1	0.727273

Next steps: Generate code with `jde_melted` View recommended plots New interactive sheet

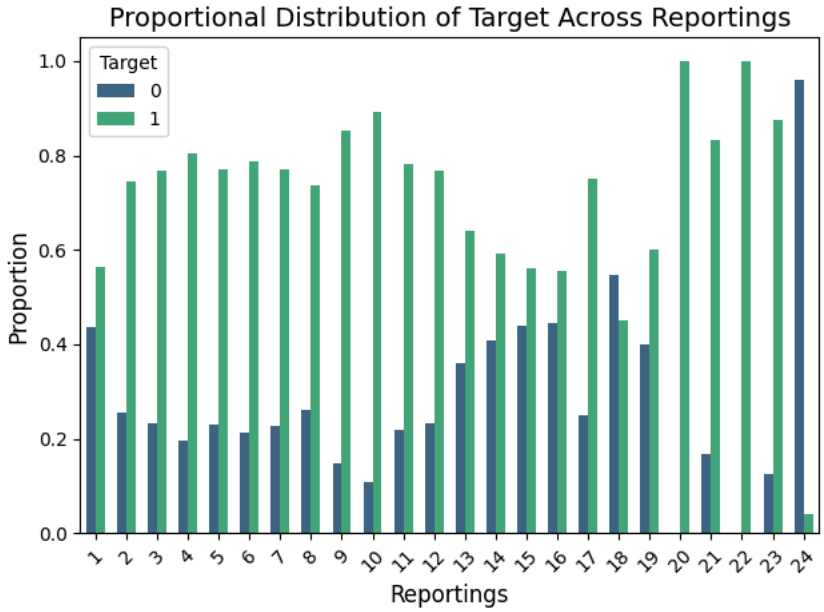
`ola.head()`

	Driver_ID	Reportings	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Gender	Ratir
0	1	3	1	28	2	23	57387.0	1	1	1715580.0	2	0	
1	2	2	0	31	2	7	67016.0	2	2	0.0	1	0	
2	4	5	1	43	2	13	65603.0	2	2	350000.0	1	0	
3	5	3	1	29	0	9	46368.0	1	1	120360.0	1	0	
4	6	5	0	31	1	11	78728.0	3	3	1265000.0	2	1	

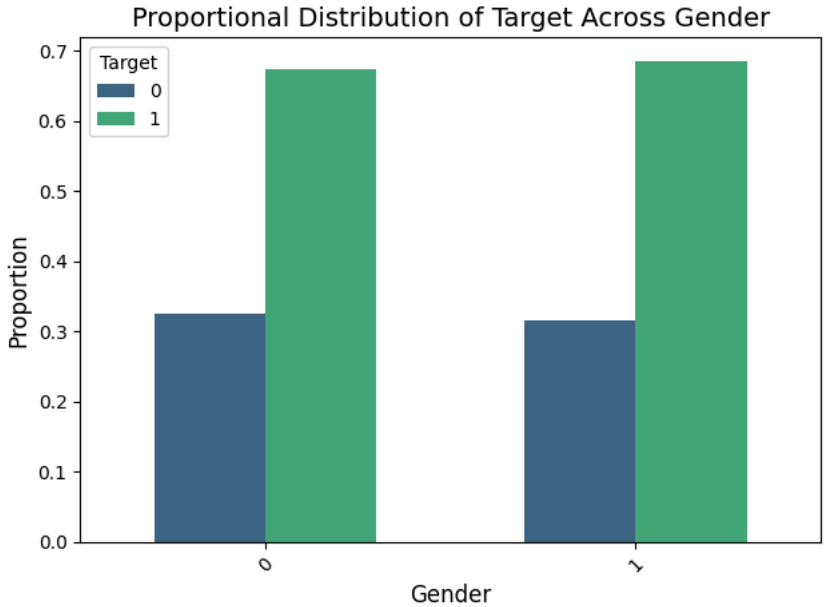
Next steps: Generate code with `ola` View recommended plots New interactive sheet

```
# Calculate normalized proportions for each Joining Designation

new_cols= ['Reportings','Gender','Education_Level','Joining Designation','Grade','Quarterly Rating','Rating_Rise','Income_Ri
for i in new_cols:
  df_new = pd.crosstab(ola[i], ola['Target'], normalize='index').reset_index()
  print(df_new)
  # Melt the dataframe for easier plotting with seaborn
  df_melted = df_new.melt(id_vars=i, var_name='Target', value_name='Proportion')

  # Plot using seaborn
  #plt.figure(figsize=(10,6))
  sns.barplot(data=df_melted, x=i, y='Proportion', hue='Target', palette='viridis',width=0.6)

  # Add labels and title
  plt.title(f"Proportional Distribution of Target Across {i}", fontsize=14)
  plt.xlabel(i, fontsize=12)
  plt.ylabel("Proportion", fontsize=12)
  plt.xticks(rotation=45)
  plt.legend(title="Target")
  plt.tight_layout()

  plt.show()
```

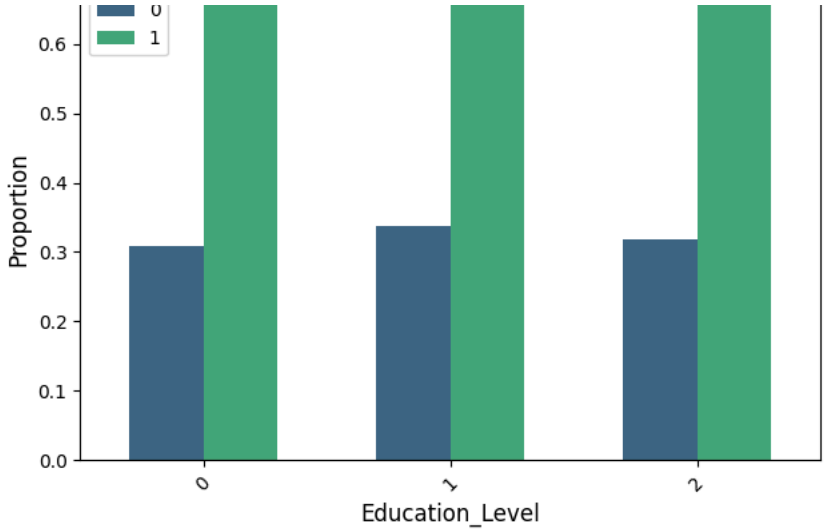
Target	Reportings	0	1
0	1	0.436464	0.563536
1	2	0.256158	0.743842
2	3	0.231939	0.768061
3	4	0.195918	0.804082
4	5	0.229773	0.770227
5	6	0.213198	0.786802
6	7	0.227941	0.772059
7	8	0.262136	0.737864
8	9	0.146789	0.853211
9	10	0.107143	0.892857
10	11	0.218182	0.781818
11	12	0.232558	0.767442
12	13	0.360000	0.640000
13	14	0.408163	0.591837
14	15	0.440000	0.560000
15	16	0.444444	0.555556
16	17	0.250000	0.750000
17	18	0.548387	0.451613
18	19	0.400000	0.600000
19	20	0.000000	1.000000
20	21	0.166667	0.833333
21	22	0.000000	1.000000
22	23	0.125000	0.875000
23	24	0.960699	0.039301



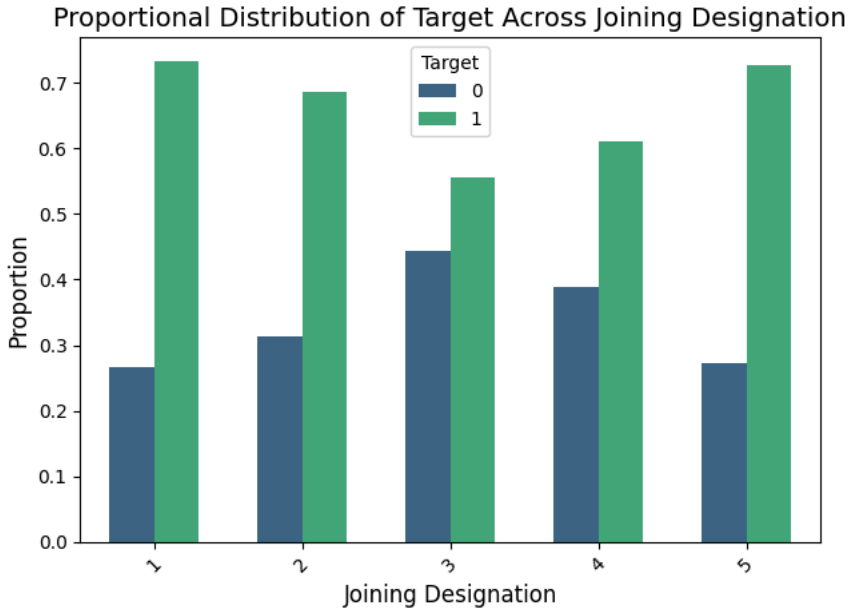
Target	Gender	0	1
0	0	0.325747	0.674253
1	1	0.314872	0.685128



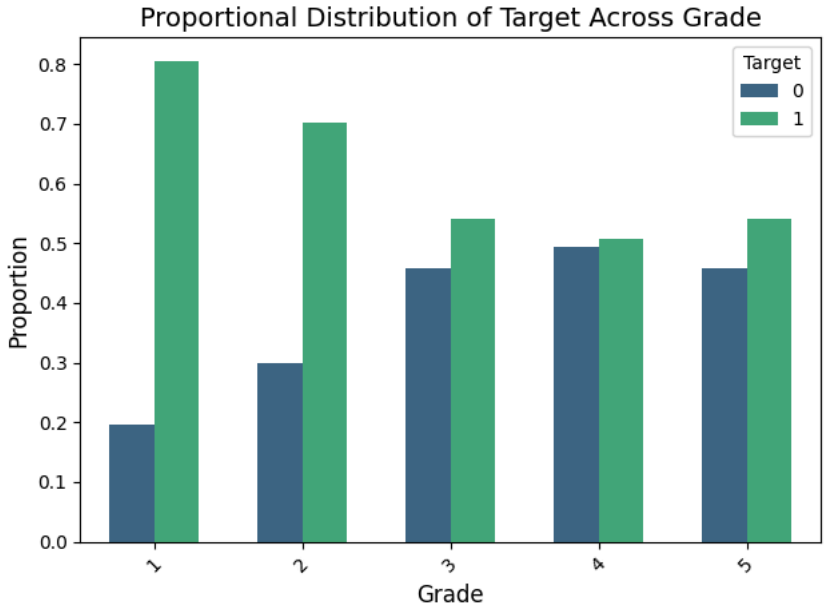
Target	Education_Level	0	1
0	0	0.308673	0.691327
1	1	0.337107	0.662893
2	2	0.317955	0.682045



Target	Joining Designation	0	1
0	1	0.267057	0.732943
1	2	0.312883	0.687117
2	3	0.444219	0.555781
3	4	0.388889	0.611111
4	5	0.272727	0.727273



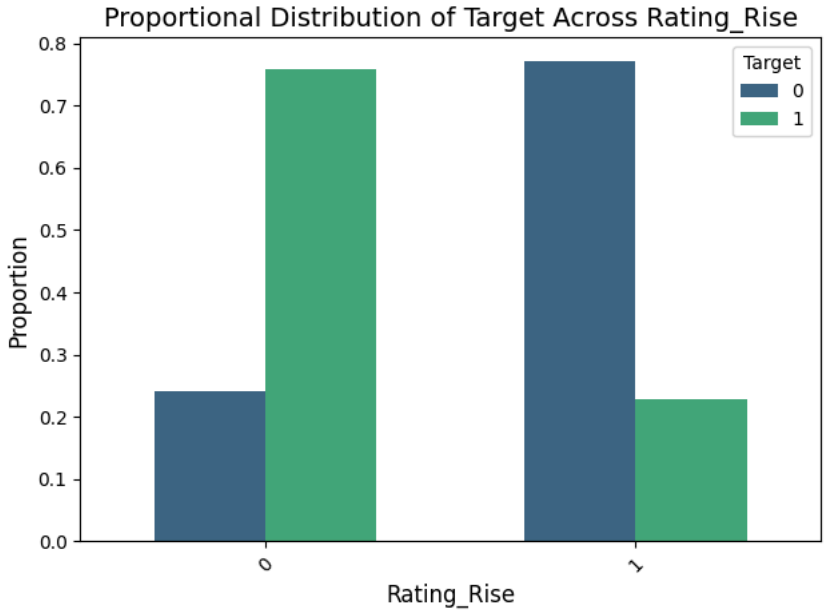
Target	Grade	0	1
0	1	0.195682	0.804318
1	2	0.298246	0.701754
2	3	0.459069	0.540931
3	4	0.492754	0.507246
4	5	0.458333	0.541667



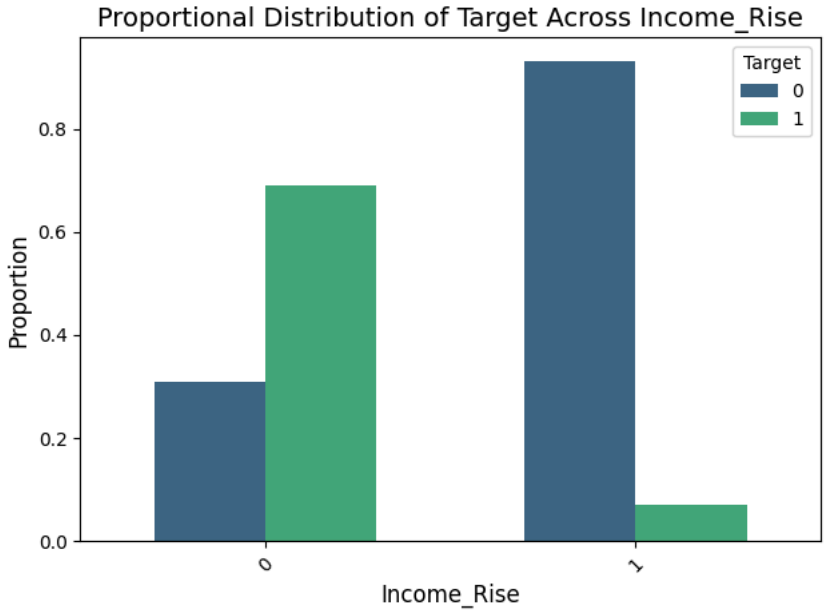
Target	Quarterly Rating	0	1

```
0                    1  0.178899  0.821101
1                    2  0.596685  0.403315
2                    3  0.833333  0.166667
3                    4  0.906542  0.093458
```

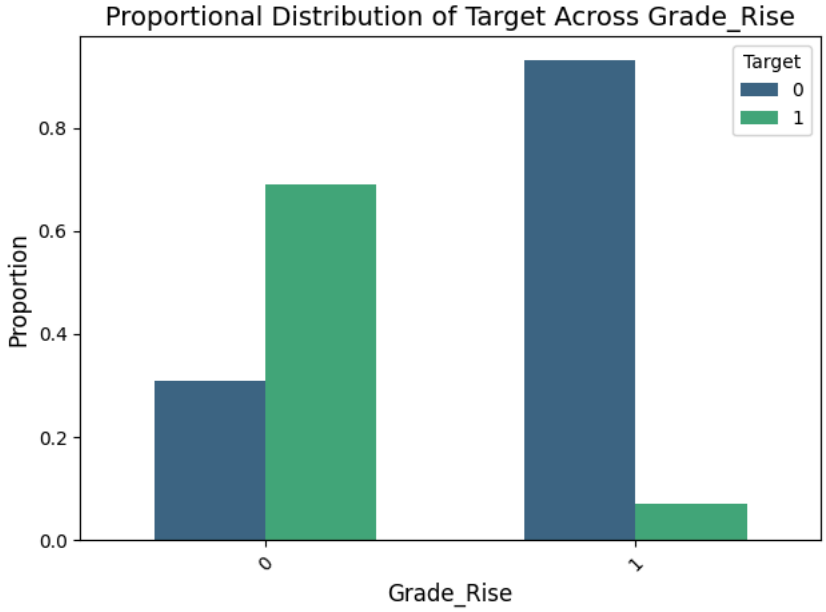

```
Target  Rating_Rise        0        1
0                 0  0.24172  0.75828
1                 1  0.77095  0.22905
```




```
Target  Income_Rise        0        1
0                 0  0.310094  0.689906
1                 1  0.930233  0.069767
```



Target	Grade_Rise	0	1
0	0	0.310094	0.689906
1	1	0.930233	0.069767



- Upon analyzing the data, we found no significant relationship between driver churn and their Gender or Education Level. Both variables show similar churn rates across different groups
- Drivers who joined the organization at Grades 3, 4, and 5 and Joining Designations 3,4,5 exhibit a lower churn rate compared to those in other grades. This suggests that these grades are associated with a higher level of retention, making drivers in these positions less likely to leave the organization.
- Drivers with high quarterly ratings, grade increases, and income rises are less likely to leave the organization.

```
ola.head()
```

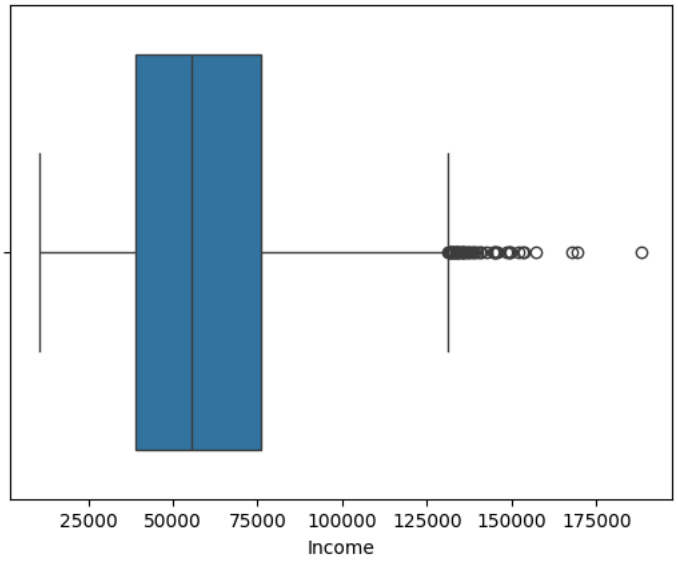
	Driver_ID	Reportings	Target	Age	Education_Level	City	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	Gender	Ratir
0	1	3	1	28	2	23	57387.0	1	1	1715580.0	2	0	
1	2	2	0	31	2	7	67016.0	2	2	0.0	1	0	
2	4	5	1	43	2	13	65603.0	2	2	350000.0	1	0	
3	5	3	1	29	0	9	46368.0	1	1	120360.0	1	0	
4	6	5	0	31	1	11	78728.0	3	3	1265000.0	2	1	

Next steps: Generate code with ola View recommended plots New interactive sheet

Double-click (or enter) to edit

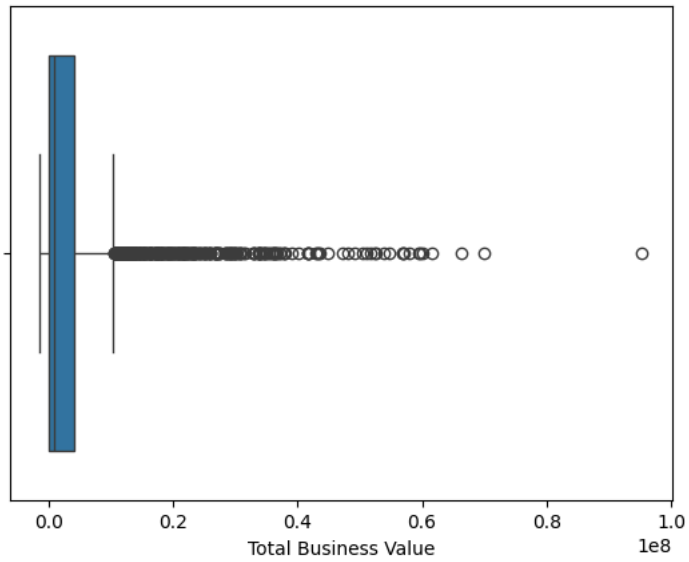
```
sns.boxplot(x=ola['Income'])
```

```
<Axes: xlabel='Income'>
```




```
sns.boxplot(x=ola['Total Business Value'])
```

```
<Axes: xlabel='Total Business Value'>
```



Total Business Value has high outliers.

ENSEMBLE LEARNING:

Data Prepration:- The Trade-Off In general while choosing a model, we might choose to look at precision and recall scores and choose while keeping the follwing trade-off on mind : -

If we prioritize precision, we are going to reduce our false positives. This may be useful if our targeted retention strategies prove to be expensive. We don't want to spend unnecessarily on somebody who is not even going to leave in the first place. Also, it might lead to uncomfortable situation for the employee themselves if they are put in a situation where it is assumed that they are going to be let go/ going to leave. • If we prioritize recall, we are going to reduce our false negatives. This is useful since usually the cost of hiring a new person is higher than retaining n experienced person. So, by reducing false negatives, we would be able to better identify those who are actually going to leave and try to retain them by appropriate measures (competitve remuneration, engagement program, etc).

```
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
```

```
# splitting the dataset into train,validatation and test samples.
```

```
X_train_val, X_test, y_train_val, y_test = train_test_split(ola.drop('Target', axis=1), ola['Target'], test_size=0.2, random
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=0.25, random_state=42)
```

```
y_train.value_counts()
```

Target	count
1	955
0	473

dtype: int64

There is a huge imbalance in the data, balancing the data before proceeding further.

```
X_sm, y_sm = SMOTE(random_state=42).fit_resample(X_train, y_train)
y_sm.value_counts()
```

Target	count
0	955
1	955

dtype: int64

```
y_val.value_counts()
```

Target	count
1	334
0	142

dtype: int64

```
#scaling the data, using X_train instead of X_sm , to weighted_class sampling instead of SMOTE sample

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)


from sklearn.model_selection import RandomizedSearchCV
from sklearn.utils import class_weight
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,classification_report


random_forest = RandomForestClassifier(random_state=42,class_weight='balanced')

params= {'max_depth':[2,3,4,5,6,7,8,9],'n_estimators':[50,100,200]}

random_search= RandomizedSearchCV(random_forest,params,cv=3,scoring='f1')
random_search.fit(X_train_scaled,y_train)

print(f"Best Parameters , {random_search.best_params_}")

y_pred= random_search.predict(X_val_scaled)

print(classification_report(y_val,y_pred))
print(confusion_matrix(y_val,y_pred))
```

```
Best Parameters , {'n_estimators': 200, 'max_depth': 8}
              precision    recall  f1-score   support

           0       0.85      0.89      0.87       142
           1       0.95      0.93      0.94       334

    accuracy                           0.92       476
   macro avg       0.90      0.91      0.91       476
weighted avg       0.92      0.92      0.92       476

[[127  15]
 [ 22 312]]
```

- The Random Forest With Class Weighting method out of all predicted 0 the measure ofcorrectly predicted is 85%, and for 1 it is 95% (Precision). The Random Forest With Class Weighting method out of all actual 0 the measure of correctly predicted is 89%, and for 1 it is 93%(Recall).

```
# Predicting on test data with  best parameters
rf = RandomForestClassifier(max_depth=8,n_estimators= 200,random_state=42,class_weight='balanced')
rf.fit(X_train_scaled, y_train)
y_pred_test= rf.predict(X_test_scaled)

print(classification_report(y_test,y_pred_test))
print(confusion_matrix(y_test,y_pred_test))
```

```
              precision    recall  f1-score   support

           0       0.88      0.89      0.88       150
           1       0.95      0.94      0.94       327

    accuracy                           0.92       477
   macro avg       0.91      0.91      0.91       477
weighted avg       0.92      0.92      0.92       477

[[133  17]
 [ 19 308]]
```

- The Random Forest With Class Weighting method out of all predicted 0 the measure ofcorrectly predicted is 88%, and for 1 it is 95% (Precision). The Random Forest With Class Weighting method out of all actual 0 the measure of correctly predicted is 89%, and for 1 it is 94%(Recall).

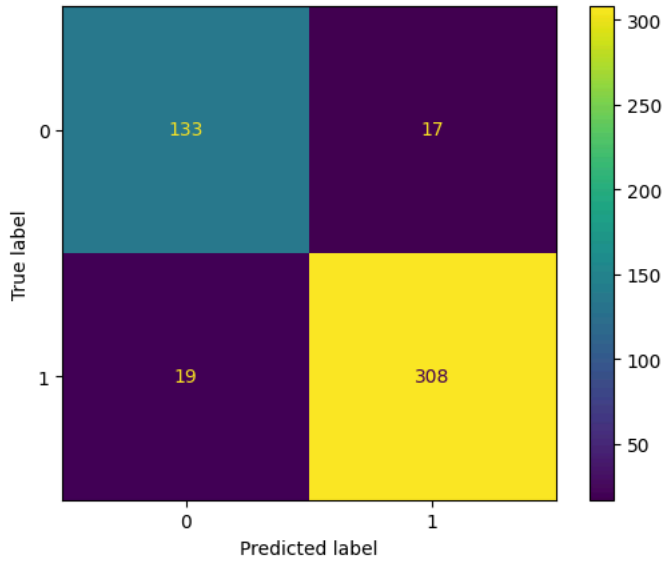
```
# XGBoosting and Gradient Boosting Classifier

import time
from xgboost import XGBClassifier
from sklearn.ensemble import GradientBoostingClassifier

# XGBoosting

start_time = time.time()
xgb_model = XGBClassifier(random_state=10)
xgb_model.fit(X_train_scaled, y_train)
xgb_train_time = time.time() - start_time

#Gradient Boosting Classifier

start_time = time.time()
gb_model = GradientBoostingClassifier(random_state=10)
gb_model.fit(X_train_scaled, y_train)
gb_train_time = time.time() - start_time

# Output the training times
print(f"XGBoost training time: {xgb_train_time} seconds")
print(f"GBC training time: {gb_train_time} seconds")

# TODD: Evaluate XGBoost and GradientBoostingClassifier on the test set
xgb_predictions = xgb_model.predict(X_test_scaled)
gb_predictions = gb_model.predict(X_test_scaled)

#Confustion matrix and Classification report


print(classification_report(y_test,xgb_predictions))
print(confusion_matrix(y_test,xgb_predictions))

print(classification_report(y_test,gb_predictions))
print(confusion_matrix(y_test,gb_predictions))
```

```
XGBoost training time: 0.08983135223388672 seconds
GBC training time: 0.4865410327911377 seconds
              precision    recall  f1-score   support

           0       0.95      0.95      0.95       150
           1       0.98      0.98      0.98       327

    accuracy                           0.97       477
   macro avg       0.96      0.96      0.96       477
weighted avg       0.97      0.97      0.97       477

[[142   8]
 [  7 320]]
              precision    recall  f1-score   support

           0       0.94      0.90      0.92       150
           1       0.96      0.98      0.97       327

    accuracy                           0.95       477
   macro avg       0.95      0.94      0.94       477
weighted avg       0.95      0.95      0.95       477

[[135  15]
 [  8 319]]
```

- XGBoostingClassifier method out of all predicted 0 the measure of correctly predicted is 95%, and for 1 it is 98%(Precision) and could acheive a recall of 95% on class 0 and 98% in class 1.
- GradientBoostingClassifier method out of all predicted 0 the measure of correctly predicted is 94%, and for 1 it is 96%(Precision) and could acheive a recall of 90% on class 0 and 98% in class 1.

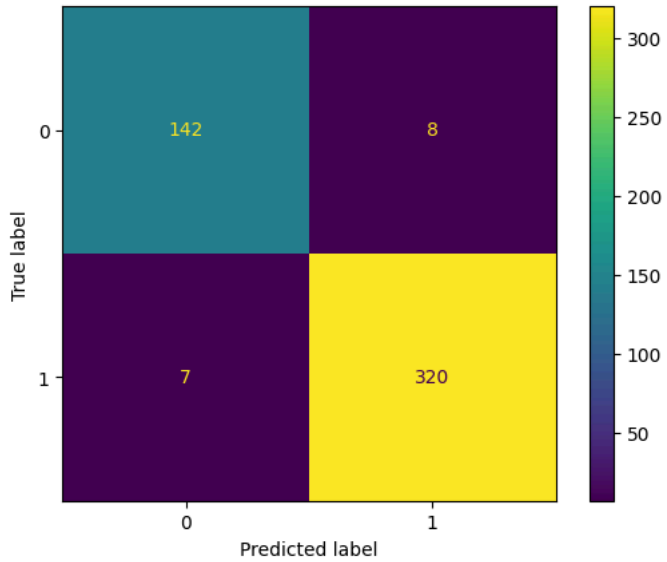
```
from sklearn.metrics import ConfusionMatrixDisplay
print("Confustion matrix from Random Forest Predictions")
ConfusionMatrixDisplay(confusion_matrix(y_test,y_pred_test)).plot();
```

```
Confustion matrix from Random Forest Predictions
```



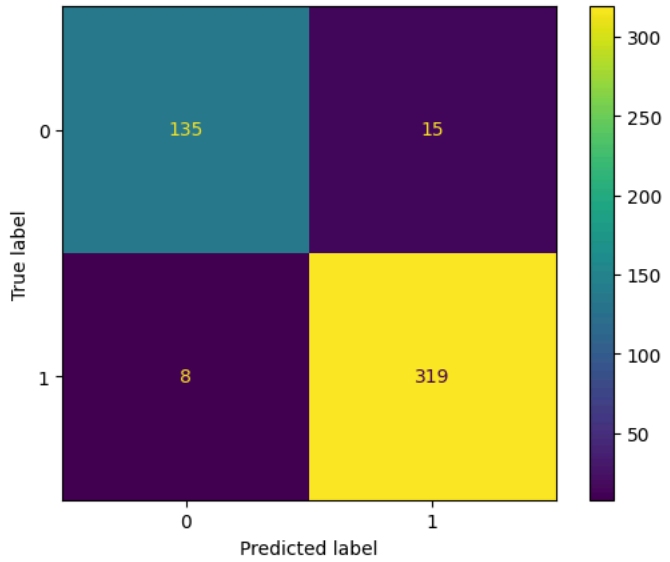

```
print("Confustion matrix from XGBoostingClassifier  Predictions")
ConfusionMatrixDisplay(confusion_matrix(y_test,xgb_predictions)).plot();
```

```
Confustion matrix from XGBoostingClassifier  Predictions
```

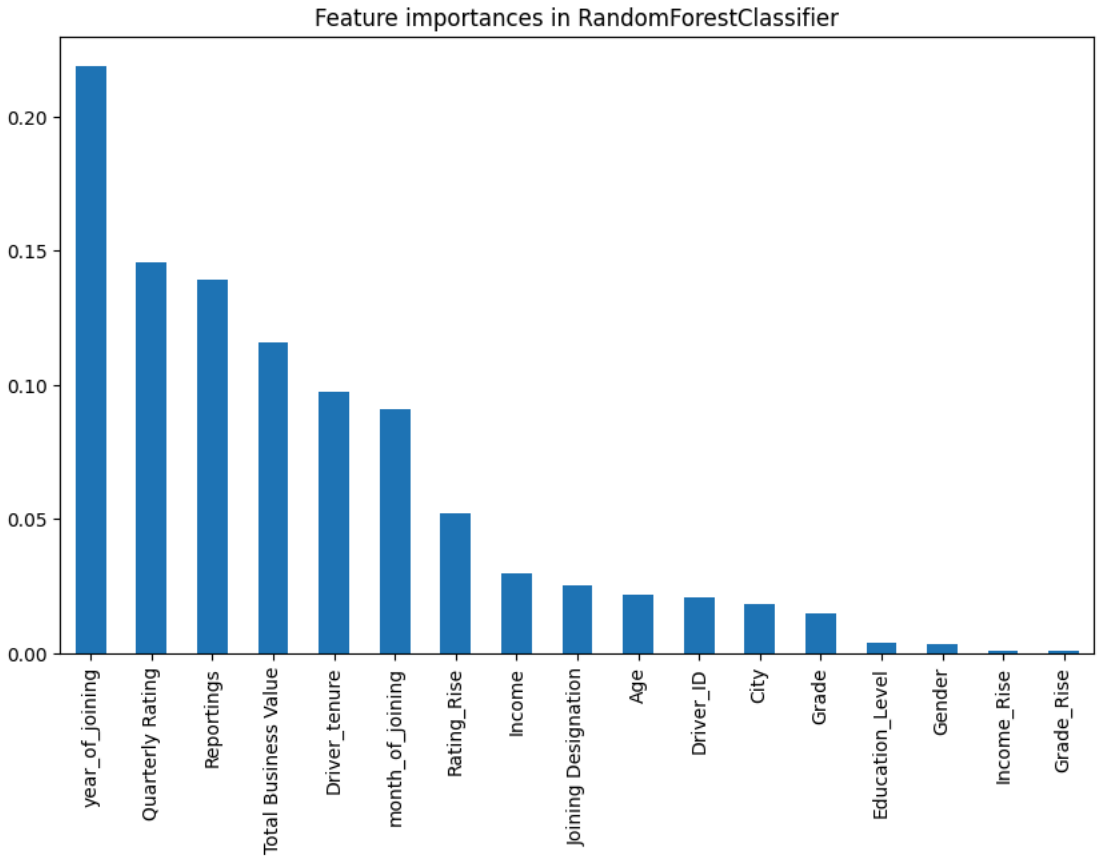


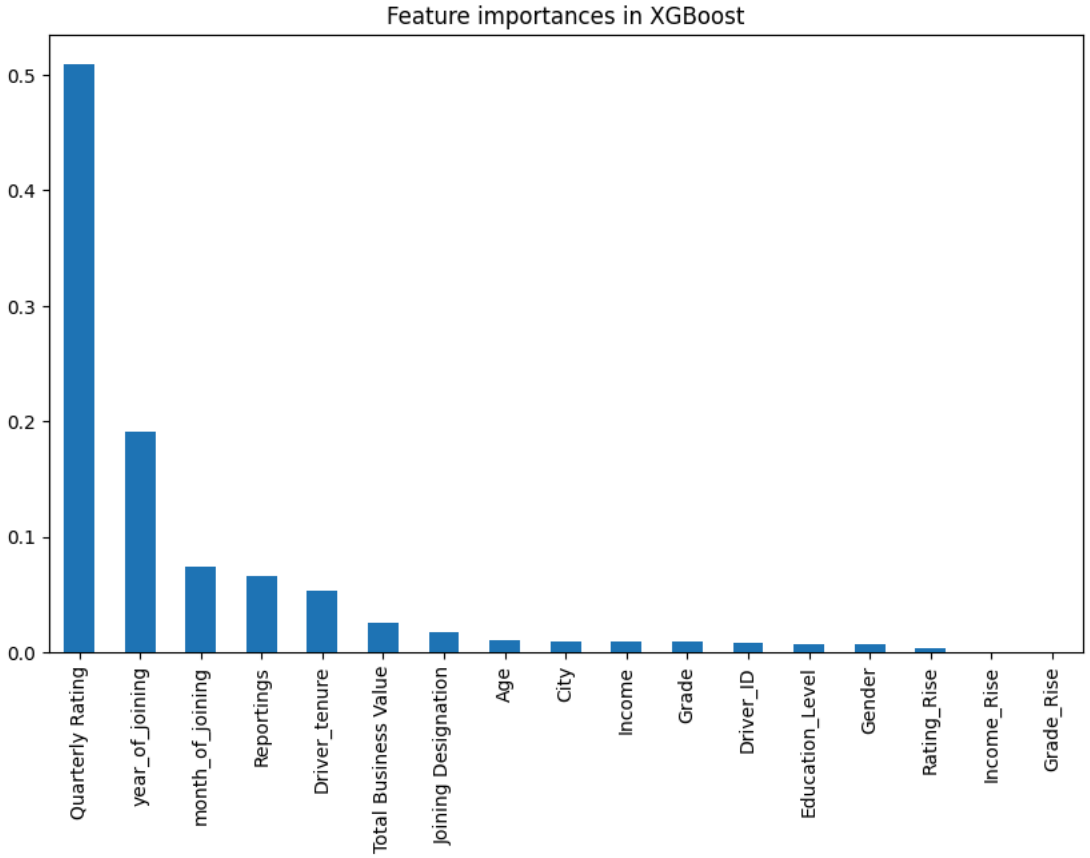

```
print("Confustion matrix from GradientBostingClassifier Predictions")
ConfusionMatrixDisplay(confusion_matrix(y_test,gb_predictions)).plot();
```

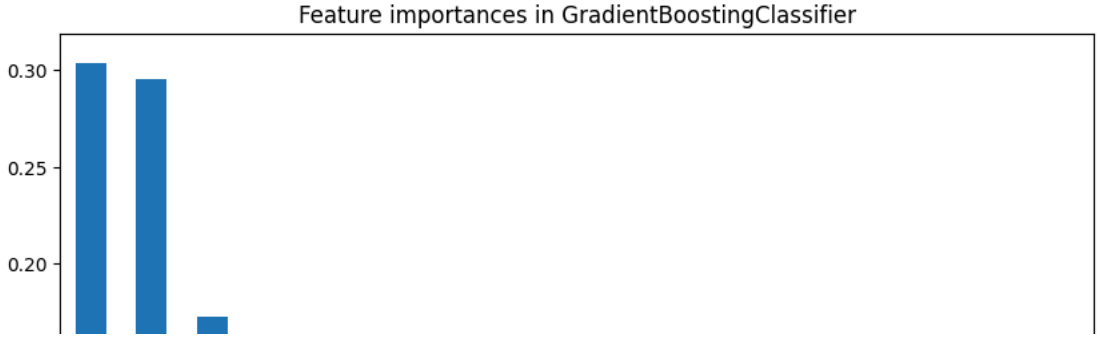
Confustion matrix from GradientBostingClassifier Predictions

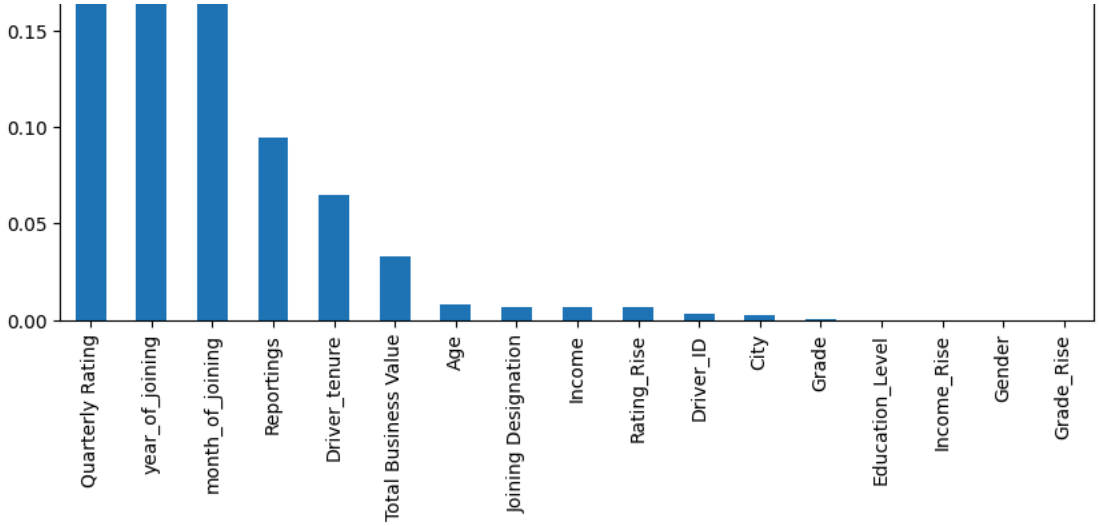



```
# Create a series containing feature importances from the model and feature names from the training data
feature_importances1 = pd.Series(rf.feature_importances_, index=ola.drop('Target',axis=1).columns).sort_values(ascending=Fal
feature_importances2 = pd.Series(xgb_model.feature_importances_, index=ola.drop('Target',axis=1).columns).sort_values(ascenc
feature_importances3 = pd.Series(gb_model.feature_importances_, index=ola.drop('Target',axis=1).columns).sort_values(ascendi

# Plot a simple bar chart
plt.figure(figsize=(10,6))

plt.title("Feature importances in RandomForestClassifier")
feature_importances1.plot.bar()
plt.show()

plt.figure(figsize=(10,6))
plt.title("Feature importances in XGBoost")
feature_importances2.plot.bar()
plt.show()

plt.figure(figsize=(10,6))
plt.title("Feature importances in GradientBoostingClassifier")
feature_importances3.plot.bar()
plt.show()
```


Feature importances in RandomForestClassifier
0.20
0.15
0.10
0.05
0.00
year_of_joining
Quarterly Rating
Reportings
Total Business Value
Driver_tenure
month_of_joining
Rating_Rise
Income
Joining Designation
Age
Driver_ID
City
Grade
Education_Level
Gender
Income_Rise
Grade_Rise

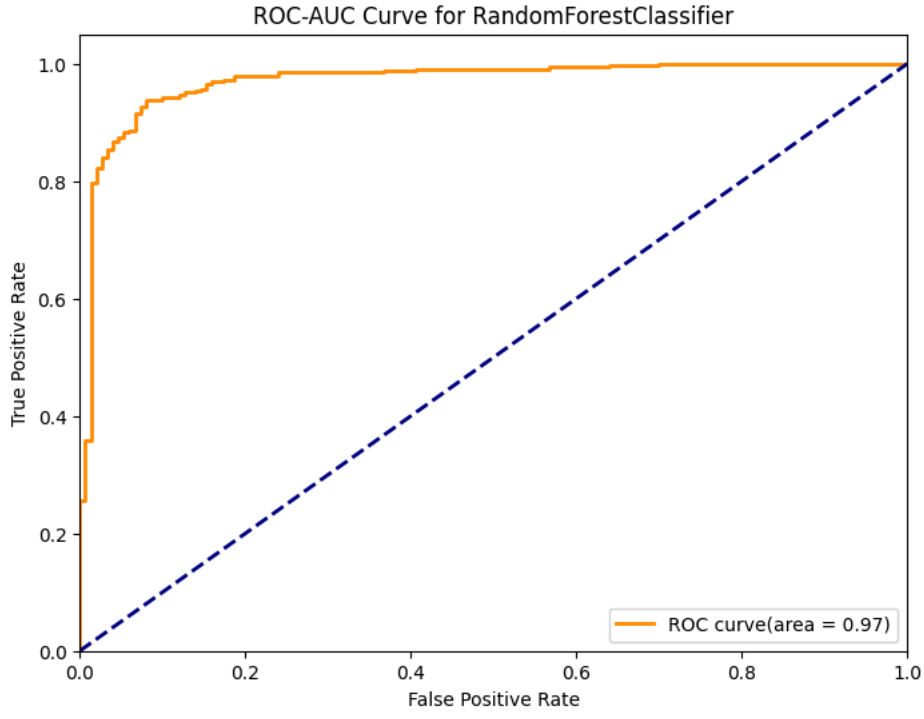

Feature importances in XGBoost
0.5
0.4
0.3
0.2
0.1
0.0
Quarterly Rating
year_of_joining
month_of_joining
Reportings
Driver_tenure
Total Business Value
Joining Designation
Age
City
Income
Grade
Driver_ID
Education_Level
Gender
Rating_Rise
Income_Rise
Grade_Rise


Feature importances in GradientBoostingClassifier
0.30
0.25
0.20



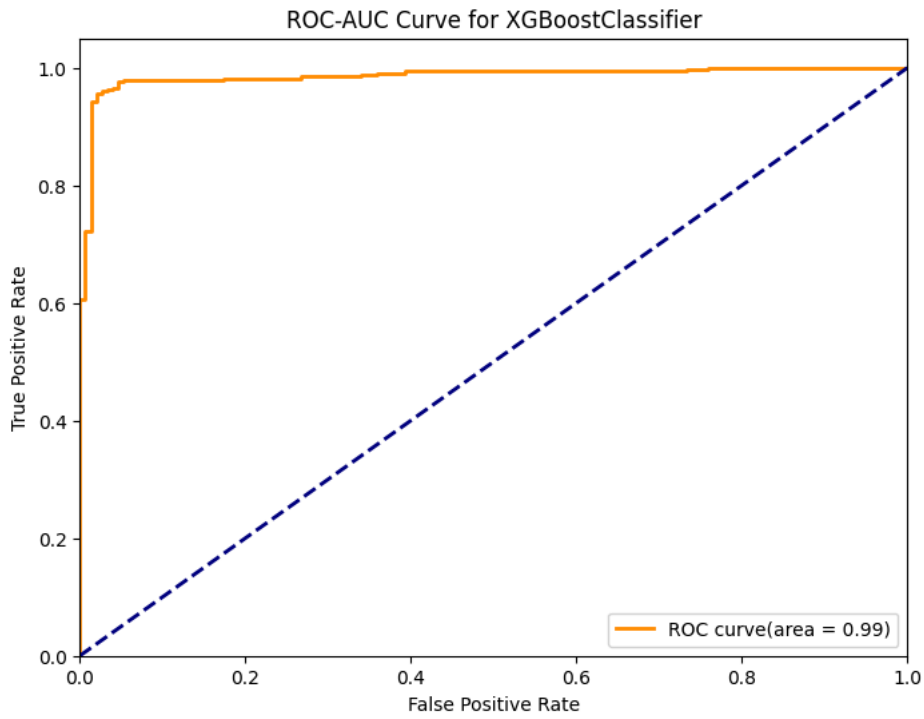
Quarterly Rating, Reportings , Year of Joining are found to be the top three important features.

```
# ROC-AUC curve for all the classifiers

from sklearn.metrics import roc_curve, roc_auc_score, auc

y_prob = rf.predict_proba(X_test_scaled)[:, 1]  # Probability for class 1

# Calculate ROC curve and AUC score
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

# Plot ROC-AUC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve(area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')  # Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-AUC Curve for RandomForestClassifier')
plt.legend(loc="lower right")
plt.show()

# Print AUC score
print(f'AUC Score: {roc_auc:.2f}')
```



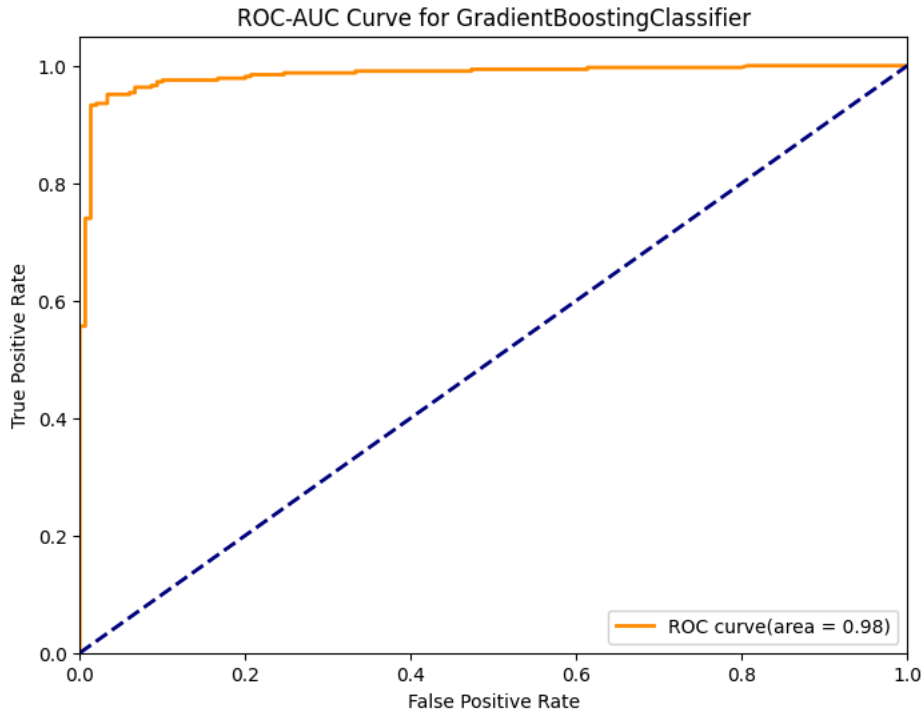

```
AUC Score: 0.97
```

```
y_prob_xgb = xgb_model.predict_proba(X_test_scaled)[:, 1]  # Probability for class 1

# Calculate ROC curve and AUC score
fpr, tpr, thresholds = roc_curve(y_test, y_prob_xgb)
roc_auc = auc(fpr, tpr)

# Plot ROC-AUC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve(area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')  # Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-AUC Curve for XGBoostClassifier')
plt.legend(loc="lower right")
plt.show()

# Print AUC score
print(f'AUC Score: {roc_auc:.2f}')
```




```
AUC Score: 0.99
```

```
y_prob_gb = gb_model.predict_proba(X_test_scaled)[:, 1]  # Probability for class 1

# Calculate ROC curve and AUC score
fpr, tpr, thresholds = roc_curve(y_test, y_prob_gb)
roc_auc = auc(fpr, tpr)

# Plot ROC-AUC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve(area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')  # Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-AUC Curve for GradientBoostingClassifier')
plt.legend(loc="lower right")
plt.show()

# Print AUC score
print(f'AUC Score: {roc_auc:.2f}')
```




```
AUC Score: 0.98
```

Insight:

Data Collection Period:

Data was gathered over an eight-year period, spanning January 2013 to December 2020. Churn Rate:

A significant churn rate was observed, with 1616 out of 2381 drivers (approximately 68%) leaving the organization. Retention Factors:

Drivers who received positive performance evaluations, salary increases, and promotions were more likely to remain with the company. Geographic Distribution:

Cities 20, 29, and 22 were the primary sources of new driver hires. Model Performance:

XGBoost demonstrated exceptional performance in predicting driver retention, achieving a precision of 95% and a recall of 98% on the test dataset.

Recommendations:

Recommendations:

Enhance Retention Efforts:

Reward and Recognition: Implement a robust reward and recognition program to acknowledge and appreciate top-performing drivers. Career Development: Offer opportunities for professional growth and development, including training programs and mentorship. Competitive Compensation: Ensure compensation packages remain competitive to attract and retain talent. Optimize Recruitment Strategies:

Target Specific Regions: Focus recruitment efforts on cities 20, 29, and 22, where there is a proven pool of potential drivers. Leverage Referral Programs: Encourage existing drivers to refer their colleagues, tapping into their networks for qualified candidates. Utilize Predictive Analytics:

Deploy XGBoost or similar models: Continuously monitor driver behavior and identify early warning signs of churn using predictive analytics. Proactively address risks: Take proactive steps to address potential churn factors before drivers leave. Improve Employee Experience:

Gather feedback: Conduct regular surveys and feedback sessions to understand driver needs and concerns. Address pain points: Implement changes to improve the driver experience, such as enhancing communication channels or providing better support services. Data-Driven Decision Making:

Leverage analytics insights: Use data-driven insights to inform strategic decisions and optimize operations. Monitor key metrics: Track key performance indicators (KPIs) related to driver retention, satisfaction, and efficiency. By implementing these recommendations, the organization can significantly improve driver retention, reduce costs associated with turnover, and enhance overall operational efficiency.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.